

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

BACHELOR THESIS

Tamar Noselidze

Adversarial Examples Against Vision Transformers

Department of Theoretical Computer Science and Mathematical Logic

Supervisor of the bachelor thesis: Doc. Mgr. Martin Pilát, Ph.D.

Study programme: Computer Science with a
specialization in Artificial
Intelligence

Prague 2025

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

First and foremost, I would like to express my gratitude to my thesis supervisor, doc. Mgr. Martin Pilát, Ph.D., for his immense help and support throughout the many months of our work together. His guidance has been incredibly insightful and consistently encouraging, which truly makes him an exceptional teacher and supervisor.

I also dedicate a heartfelt thank you to my family, especially my parents, who have been my biggest supporters in every stage of life.

Finally, I want to thank my closest friends, for always having my back and making all the difficult times studying enjoyable.

Title: Adversarial Examples Against Vision Transformers

Author: Tamar Noselidze

Department: Department of Theoretical Computer Science and Mathematical Logic

Supervisor: Doc. Mgr. Martin Pilát, Ph.D., Department of Theoretical Computer Science and Mathematical Logic

Abstract: Deep learning models have been remarkably successful in image classification tasks, although they still remain vulnerable to adversarial examples. These are modified input images, designed in a special way to fool victim models into making incorrect predictions. Among adversarial methods, we focus on patch-based attacks as they pose security risks, due to their physical applicability in real-world scenarios. This thesis explores the effectiveness of patch-based adversarial attacks against Vision Transformers, as well as their transferability across model architectures. We begin with evaluating the effectiveness of several white-box gradient-based methods on Vision Transformers and compare it to Convolutional Neural Networks. We then replicate an existing patch-based attack, called G-Patch and further build on it. Our contributions include a novel Mini-Patch method with three deployment strategies: random placement, corner-point and token-replacement. We carried out extensive experiments in each setting and learned that corner-point patches perform comparably to state-of-the-art methods. We further investigate transferability in both G-Patch and Mini-Patch settings, which revealed that mixed-architecture ensembles help with generalizability of adversarial examples. Overall, our work provides insights into ViT model vulnerabilities and how they compare to CNNs, as well as attack transferability.

Keywords: Adversarial Attacks, Vision Transformers, Patch Attacks, Convolutional Neural Networks

Contents

Introduction	7
1 Theoretical Background	9
1.1 Model Architectures	9
1.1.1 Convolutional Neural Networks	9
1.1.2 Vision Transformers	10
1.2 Adversarial Attacks	13
1.3 Adversarial Transferability	15
2 Experiments with existing attacks	17
2.1 Code Setup	17
2.2 Fast Gradient Sign Method (FGSM)	18
2.3 Projected Gradient Descent (PGD)	22
3 Random Position Patch Attack	25
3.1 Overview of the Paper	25
3.2 Our Implementation	26
3.2.1 Generator	26
3.2.2 Deployment	26
3.2.3 Discriminators	27
3.2.4 Adversarial Loss	28
3.2.5 Training Process	28
3.2.6 Evaluation	29
3.3 Experiment Results	30
3.3.1 64×64 Patch Results	30
3.3.2 80×80 Patch Results	34
3.3.3 Comparing to original paper	37
3.4 Transferability	39
3.4.1 Intra-Family Transferability	39
3.4.2 Inter-Family Transferability	43
3.4.3 Summary	50
4 Mini Patch Attacks	51
4.1 Random Position Mini Patches	51
4.2 Exploiting Critical Points	55
4.2.1 "Corner" points	55
4.2.2 Critical points within token patches	59
4.3 Transferability	62
Conclusion	67
Bibliography	69
List of Figures	71
List of Tables	75

A	Attachments	77
A.1	CleverHans	77
A.2	Random_Position_Patch	77
A.3	User Documentation	77
A.4	Link to the GitHub Repository	77

Introduction

Machine learning has revolutionized the field of image classification and object detection, enabling machines to achieve performance levels comparable to, and in some cases even surpassing, human ability. In particular, the rise of neural networks resulted in an incredibly powerful tool – **Convolutional Neural Networks** (CNNs) – which remained dominant in image processing tasks for many years. However, not even a decade ago, **Vision Transformers** (ViTs) emerged as a state-of-the-art architecture. Since then, ViTs have outperformed traditional CNNs in several computer vision tasks, mainly due to the self-attention mechanism, which allows them to capture relationships between distant parts of an image.

Despite their success, ViT models, like CNNs, are still vulnerable to **adversarial examples** – i.e., input images that have been subtly modified to fool the model into making incorrect classifications. This vulnerability has led to extensive research on adversarial attacks, aiming to expose the weaknesses of machine learning models. Most of these attacks focus on adding small *perturbations* to input images, making them indistinguishable from the original to the human eye, yet deceiving to the victim model.

There also exist so-called **patch attacks**, which, unlike traditional perturbations, modify a specific region of the image with a noticeable patch. These attacks are not only effective, but also more practical in real-world settings, as they do not require pixel-level precision or knowledge of full image content.

The real-life application of adversarial attacks poses significant security risks, especially in cases where image classification is quite important, such as autonomous driving, healthcare diagnostics, and surveillance systems. Therefore, it is rather crucial, more than interesting, to research the topic and contribute to strengthening the existing models against such perturbations.

While much research has been conducted on adversarial attacks targeting CNNs, the robustness of Vision Transformers in these settings remains less explored. This thesis is motivated by the need to better understand how ViTs respond to adversarial examples. In particular, we aim to:

- Explore existing adversarial attacks and evaluate the robustness of Vision Transformer models against them. For comparison, we also assess the vulnerabilities of CNN models under the same attacks;
- Implement a novel approach for patch-based attacks proposed in the paper "Designing Physical-World Universal Attacks on Vision Transformer" (Shao, 2024), and replicate its results;
- Propose a new patch-based attack inspired by the above-mentioned approach;
- Investigate the transferability of explored attacks both, within Vision Transformer families and across different model architectures.

Thesis Outline

- Chapter 1 provides the theoretical background necessary to understand the experiments conducted in this thesis;

- Chapter 2 presents results of several standard attacks using the CleverHans library, establishing a baseline for the subsequent experiments on patch-based attacks;
- Chapter 3 details the implementation of the paper "Designing Physical-World Universal Attacks on Vision Transformer" (Shao, 2024). Section 3.4 further explores the transferability of this attack across different model architectures;
- Chapter 4 introduces our novel approach to patch-based adversarial examples, called *Mini-Patch Attacks*. Additionally, Section 4.3 investigates the transferability of this method;
- Conclusion offers a concise yet comprehensive summary of the key findings of this work.

1 Theoretical Background

In this chapter, we provide the important theoretical background for understanding the experiments conducted in this thesis. We first describe the model architectures relevant to our work and explore the key differences between them. These differences play a crucial role in understanding how the models respond to adversarial perturbations.

Next, we describe the concept of adversarial machine learning and review related work about various existing adversarial attacks. This involves an overview of how adversarial examples are generated and why they are important in evaluating modern deep learning models.

Finally, we discuss the transferability of adversarial examples across different models, which is one of the main goals of this thesis.

1.1 Model Architectures

We begin with exploring the model architectures used in this thesis, specifically Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs). Understanding their structural differences is important for our analysis of their behavior in adversarial settings.

1.1.1 Convolutional Neural Networks

Convolutional Neural Networks (Krizhevsky et al., 2012) are a class of deep learning models mainly used for image processing tasks, such as image classification, object detection, etc. At the core of CNNs are convolution layers, which apply *filters* (also called *kernels*) to input images to extract patterns and features, for example, edges and shapes. These filters are learnable parameters that we refine during training to detect the most relevant features for a specific task.

In each convolution layer we *slide* the filter along the matrix representation of an input image, and perform the **convolution** operation. This is usually nothing more than a mere dot product of a filter and the layer’s input matrix. This, in turn, generates a very abstract representation of the initial image – a **feature map**, that then serves as an input matrix for the next convolution layer. More formally, the feature map is computed as:

$$F[m, n] = (I * K)[m, n] = \sum_i \sum_j K[i, j] I[m - i, n - j]$$

where $*$ stands for the convolution operator, F denotes the feature map matrix, while I denotes the input image and K – the kernel. The indices of rows and columns of the resulting matrix are marked with m and n respectively.

To introduce non-linearity into the model, a ReLU activation is applied element-wise to the output of convolutional layer. ReLU outputs the input directly if it is positive, otherwise, it returns zero:

$$\text{ReLU}(x) = \max(0, x)$$

where x represents the output of the convolution operation before activation (pre-activation value).

Convolutional layers of the network are typically followed by **pooling** layers to down-sample the generated feature maps. This reduces the spatial dimensions of the data, which makes the computation more efficient and also maintains the most important information at the same time. **Max pooling**, the most common type, selects the maximum value from non-overlapping rectangular regions of the feature map:

$$P[i, j] = \max_{(m, n) \in \mathcal{R}_{i, j}} F[m, n]$$

where F again, is the feature map (typically with ReLU already applied to it) and $\mathcal{R}_{i, j}$ is the region corresponding to output location (i, j) .

CNNs also usually include **fully connected** layers at the end to combine extracted features and make predictions. These are often followed by a softmax or sigmoid function, depending on whether the task is multi-class classification or binary classification.

This whole process of convolving the input and passing the result to the next layer is similar to how a *neuron* in the visual cortex responds to a stimulus, hence the name of the architecture (Convolutional *Neural* Network). To better understand the overall architecture of CNNs, we include Figure 1.1 below:

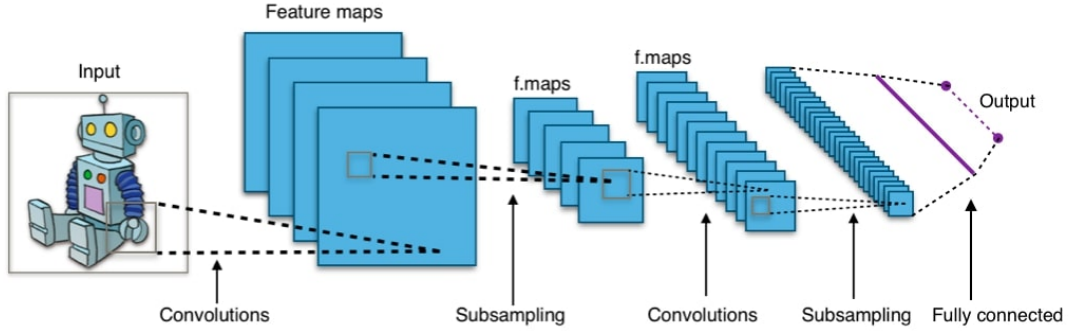


Figure 1.1 Representation of the architecture of a Convolutional Neural Network. Source: Wikimedia.

CNN architectures have evolved significantly, with popular models like VGGNet (Simonyan; Zisserman, 2015) or ResNet (He et al., 2015) pushing the boundaries of performance in benchmarks. They are very good at effectively extracting relevant features from input images, and have become the standard for many computer vision tasks, especially image classification.

1.1.2 Vision Transformers

Vision Transformers (Dosovitskiy et al., 2021) are quite different from traditional convolution-based approaches. As the name suggests, they are transformer-based architectures for image classification tasks. Their design is inspired by the transformer architecture originally developed for natural language processing (NLP), which was later adapted to also work with image data in a novel way.

The architecture of a ViT starts by splitting an input image into fixed-size **patches** (typically, 16×16 pixels). These patches are flattened into 1-dimensional

vectors and then transformed into a format the model can work with using simple linear mappings (i.e. *embeddings*). We present this step with a mathematical formula:

$$x_p = \text{Flatten}(x)W_e + b$$

where x is a patch of the input image, W_e is a learnable embedding weight matrix, and b is a bias term.

Positional embeddings are then added to these vectors to maintain information about where each patch originally was positioned in the image. This step is crucial since transformers do not naturally understand the spatial structure of images. It ensures that the model understands the relative position of the patches.

The sequence of patches, now resembling a sequence of tokens in NLP tasks, is then fed into a stack of transformer layers. Each transformer layer consists of:

1. **Multi-Head Self-Attention:** This mechanism, which is the key difference between ViTs and CNNs, enables image patches to "communicate" with each other. It essentially learns relationships between patches across the entire image. Most importantly, multi-head attention allows the model to perform this simultaneously (through multiple attention heads), which makes it efficient to capture both local and global patterns in a *parallelized* approach. Attention mechanism can be summarized as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where

- Q is a query matrix (projected from input embeddings);
- K is a key matrix;
- V is a value matrix;
- d_k is the dimensionality of the key vectors (used for scaling).

Multi-headed attention can be formulated like:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where:

- h is the number of attention heads;
 - W_i^Q, W_i^K, W_i^V are projection matrices for queries, keys, and values in the i -th head, respectively;
 - W^O is the output projection matrix after concatenating heads;
 - head_i is the output of the i -th attention head.
2. **Feed-Forward Network:** This is a fully connected feed-forward layer, which is applied to each patch embedding separately. This step introduces non-linearity and helps the model better learn complex patterns.

3. **Normalization and Residual Connections:** Layer normalization, just like in CNNs, ensures stable training. And, residual connections reduce the issues like the vanishing gradient problem, which makes it easier for the model to learn.

This complex architecture of Vision Transformers is illustrated in Figure 1.2 below:

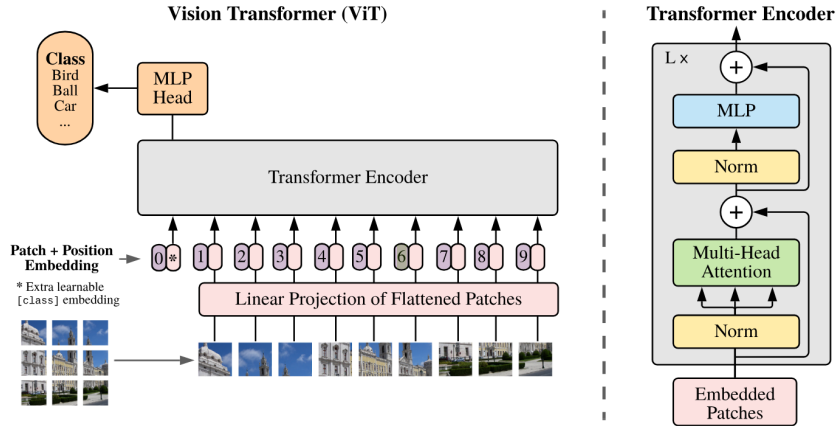


Figure 1.2 Representation of the architecture of a Vision Transformer. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence (from Dosovitskiy et al., 2021).

In recent years, ViTs have consistently shown impressive results in image classification tasks, especially when trained on large datasets. In the following section, we will discuss how and why they differ from standard deep learning models like CNNs.

ViT vs. CNN

It is important to note that CNNs use the same set of filters for processing the entire image, which makes them more effective at learning local patterns and small features, like edges and shapes. CNN filters essentially “reuse” their knowledge across the whole image, which is why this architecture learns and generalizes well, even when the training set is small.

On the other hand, ViTs are good at capturing more global, longer range relationships, due to their self-attention mechanism. Since they rely entirely on data to learn both local and global features, they naturally require larger training sets. They can also “handle” very large amounts of training data better than CNNs, and they usually *scale* more effectively, since CNNs’ performance improvements might slow down as the dataset size increases.

Therefore, CNNs are more efficient in scenarios where training data is limited, while ViTs can potentially achieve higher performance on large-scale datasets. Other major differences between these architectures will be discussed in the following sections, as we introduce concepts of adversarial machine learning and transferability.

1.2 Adversarial Attacks

Adversarial machine learning is a field that studies how machine learning models behave with input images that have been intentionally modified to fool them. It combines two main areas: adversarial attacks and adversarial defenses. These two areas are closely connected to each other – the evolution of one usually leads to progress in the other.

In this thesis, we focus on adversarial attacks, with the goal of contributing to the existing research and helping improve the overall robustness of models.

Adversarial attacks work by manipulating the input data in a way that causes machine learning models to make incorrect predictions. We can differentiate between two main types of adversarial attacks:

- In **white box attacks**, the attacker has complete knowledge of the target model, including its architecture, weights, and parameters. This allows the attacker to create precise and targeted perturbations to the input data to mislead the model.
- In **black box attacks**, the attacker has no knowledge of the model’s internal structure. They can only observe the model’s outputs for given inputs. Despite this limitation, black box attacks can still be effective, often using techniques like “query-based” attacks, where the attacker generates adversarial examples by repeatedly querying the model with slightly modified inputs.

Examples of white-box attacks include the Fast Gradient Sign Method (FGSM) (Goodfellow et al., 2015), one of the earliest and simplest approaches to adversarial examples. This work demonstrated how even small perturbations can mislead victim models. Another method is Projected Gradient Descent (PGD) (Madry et al., 2019), which further enhanced gradient-based attacks and once again demonstrated how vulnerable CNNs are to adversarial examples. These methods have become standard benchmarks for evaluating model robustness, and we discuss them in more detail in Chapter 2 of this work.

Within black-box attacks, we mention “Square Attack” (Andriushchenko et al., 2020), which, unlike FGSM or PGD, does not rely on local gradient information. Square Attack is based on a *randomized search scheme* which randomly selects small, square-shaped patches of an image at each iteration. It applies perturbations to these small squares instead of changing every pixel a little bit. This is illustrated in Figure 1.3 below.

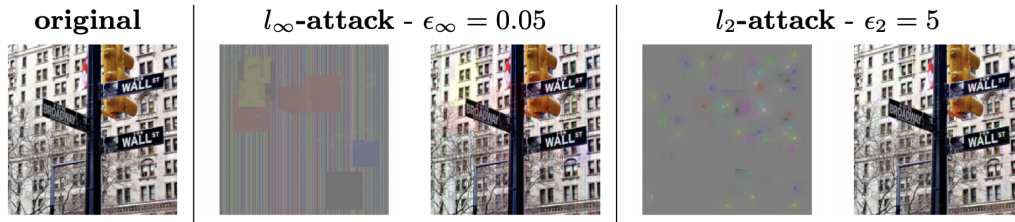


Figure 1.3 Visualization of the adversarial perturbations and examples found by the l_{∞} and l_2 versions of the Square Attack on ResNet-50 (Andriushchenko et al., 2020).

By carefully choosing where and how to apply these square patches, Square Attack can *gradually* fool the model even without knowing anything about its internal structure. This makes the attack especially powerful in real-life situations where model details are not available to the public.

This offers a nice gateway to another type of attack that will be the main focus of this thesis – **patch-based** adversarial attack. Unlike Square Attack, in patch-based attacks we usually have a *pre-generated* adversarial patch (or we create one), which is then simply placed on an image to fool the victim model. These are called **adversarial patches**, and they mislead the model into classifying the modified image as the class associated with the patch.

Adversarial patches, first introduced in the "Adversarial Patch" paper (Brown et al., 2018) can even fool models under real-world physical constraints. In their approach, the patch is first optimized in a white-box setting using an ensemble of models. This makes the patch effective even when transferred to black-box scenarios. The key idea of this work was to generate a universal patch that would fool the model, independent of the patch location within the input image. This is illustrated in Figure 1.4 below:

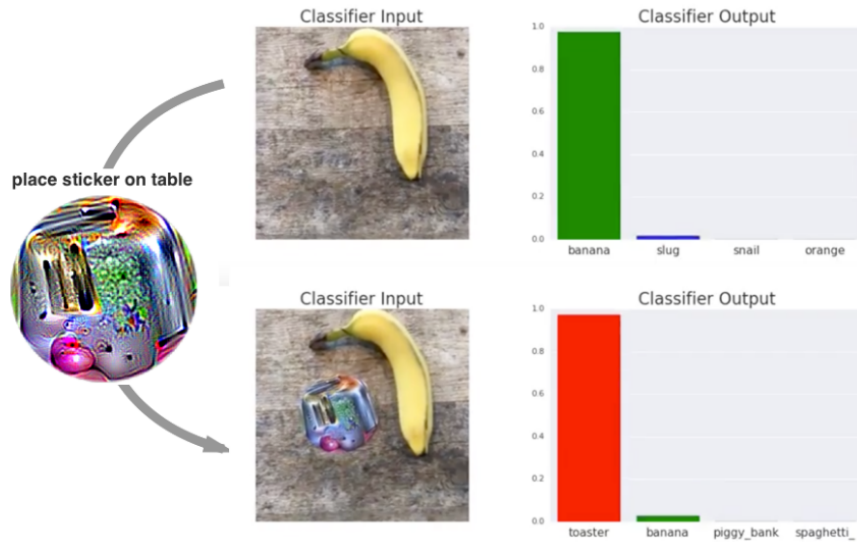


Figure 1.4 A real-world attack on VGG16, using a physical patch generated by the white-box ensemble method. When a photo of a tabletop with a banana and a notebook (top photograph) is passed through VGG16, the network reports class 'banana' with 97% confidence (top plot). If we physically place a sticker targeted to the class "toaster" on the table (bottom photograph), the photograph is classified as a toaster with 99% confidence (bottom plot) (Brown et al., 2018).

This leads us to a more recent and novel approach called "Designing Physical-World Universal Attacks on Vision Transformers" (Shao, 2024). Just like in "Adversarial Patches", the attack described in this paper aims to create adversarial examples that do not rely on their placement within the input image. However, in this case, instead of optimizing the adversarial patch itself, the paper focuses on training a **generator** that can create perturbations from random noise vectors. During training, the patch's position is fully randomized, which helps the generator to learn to produce universal adversarial examples. This paper forms the foundation of this thesis, and we explore it in more detail in Chapter 3.

Vulnerability to adversarial perturbations

Now that we have introduced the concept of adversarial attacks and explored some key examples, we briefly discuss how different architectures, specifically CNNs and ViTs, respond to them.

In CNNs, adversarial perturbations often exploit the fact that the model relies on local patterns, and attacks usually tend to be targeted at these localized, critical points. This makes it easy to distort features extracted in convolutional layers and eventually, to deceive the model.

ViTs, on the other hand, are vulnerable for different reasons. Since they process images as sequences of fixed-size patches (i.e. patch tokens), adversarial attacks can exploit not only critical parts of the overall image, but also the arrangement of these patches and the relationships between them. Additionally, because ViTs require significant computational resources and large datasets to train effectively, they may be more susceptible to attacks if the training data is insufficient or of poor quality.

Overall, while both architectures have unique strengths, their susceptibility to adversarial attacks emphasizes the need for more research about the topic. In this thesis, we aim to experiment with the adversarial methods described in previous sections. While the main focus will be on Vision Transformers, we will also include some CNN models to better understand the differences in their robustness.

1.3 Adversarial Transferability

As mentioned earlier, in addition to exploring model architectures and specific adversarial attacks, we also briefly discuss the concept of transferability, a very important feature of adversarial robustness.

Transferability refers to the phenomenon where adversarial examples generated for one machine learning model (a source model) can also mislead a different model (a target model). This is a significant aspect of adversarial robustness, as it helps analyze how well an adversarial example created for one specific model can generalize against other models within the same, and, different architectures.

However, achieving high transferability is far from easy. For example, one of the most common challenges is the model diversity – differences in model architecture, training data, hyperparameters, and regularization techniques can all influence creating universal adversarial examples. As a result, the behavior of adversarial examples across models is not always predictable, and the success rate of transferability tends to drop when models differ significantly.

Another challenge we often face is robustness variation: different models exhibit different levels of robustness to adversarial examples. Some models might be more resistant to certain types of attacks (e.g. CNNs are more robust against patch-based attacks, compared to ViT models), making the transferability less effective. This makes it harder to create universal adversarial examples that transfer well across model types.

In this thesis, we aim to investigate the transferability of patch-based attacks primarily within the Vision Transformer family. However, we also include some CNN models to evaluate cross-architecture transferability, and compare the results of fundamentally different model families.

One relevant study is "On Improving Adversarial Transferability of Vision Transformers" (Naseer et al., 2022) which suggests novel ways of crafting perturbations that generalize better across ViTs. One key suggestion the authors make is to create more "spread-out" perturbations across the image, instead of focusing on very localized areas. This is because ViTs mix information from across the entire image through their self-attention mechanism. Simple patch attacks are inherently limited in this sense. Even though they often perform quite well in a white-box setting on the specific target model, their transferability rates are very low.

Inspired by these findings and by the attack proposed in "Designing Physical-World Universal Attacks on Vision Transformers" (Shao, 2024), we will later introduce our own, novel attack method, called "Mini Patches". Details of this method are presented in Chapter 4.

2 Experiments with existing attacks

This chapter serves to establish a baseline for evaluating the novel patch-based adversarial attacks explored later in the thesis. We begin by experimenting with existing, well-known adversarial attacks using the CleverHans framework (Papernot et al., 2018). This is a widely used, open-source Python library designed to evaluate the robustness of machine learning models against adversarial examples.

Specifically, we examine two standard white-box attack methods: the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD). These provide strong baselines in adversarial robustness research and offer a reference point for assessing the effectiveness of the patch-based attacks introduced in the later chapters.

2.1 Code Setup

We conducted experiments using several pretrained models from both CNN and ViT families. Below, we describe our setup for the experiments, such as dataset preprocessing, computing environment, evaluation metrics, and model selection.

Experimental Environment

The environment setup involved the following components:

- Framework: PyTorch
- Adversarial Attack Library: CleverHans
- Hardware: NVIDIA H100 & A100 GPUs (via MetaCentrum computational nodes¹)
- Logging: Weights & Biases² for real-time tracking of our results.

Dataset

We used the ImageNetV2 dataset, which contains new data for the ImageNet benchmark. This is a new data independent of existing models, which offers a unique environment with a guarantee that accuracy scores are not affected by adaptive overfitting (Recht et al., 2019).

Data preprocessing steps included:

- Resizing images to 224×224 pixels
- Normalizing pixel values using ImageNet-specific mean and standard deviation.

¹<https://metavo.metacentrum.cz/>

²<https://wandb.ai/site>

The processed images are loaded into memory using the PyTorch DataLoader with a batch size of 32 for efficient computation.

Evaluation Metrics

To evaluate how each attack performs against our victim models, we use a simple metric, Attack Success Rate (ASR)¹:

$$ASR = \frac{\text{number of misclassified images}}{\text{total number of images}}$$

Models Evaluated

We carried out the experiments on a variety of pretrained models from ViT and CNN families:

- **CNNs:** ResNet50 and ResNet152
- **ViTs:** ViT-B/16 and ViT-L/32

These models were selected to cover both Convolutional Neural Networks and transformer-based architectures, providing a comprehensive evaluation of adversarial robustness across different model types. Within each family, we included both a standard, smaller model (e.g., ResNet50, ViT-B/16) and a larger, more complex model (e.g., ResNet152, ViT-L/32).

In the following sections, we provide a brief theoretical overview of each attack method, along with implementation details, and results observed across CNN and ViT models.

2.2 Fast Gradient Sign Method (FGSM)

Fast Gradient Sign Method (Goodfellow et al., 2015) is one of the most popular and simplest adversarial techniques, which, as the name suggests, relies on the gradients of the input image.

The attack aims to perturb the input data by adding a small amount of noise based on the gradient of the loss function (for example, mean-squared error or categorical cross-entropy) with respect to the input image. Within FGSM, we differentiate between two types of attacks: untargeted and targeted.

- **Untargeted:** in this framework we do not specify a target value and attack is considered successful if the model simply misclassifies the image – i.e., predicts any label other than the true class.
- **Targeted:** in this setting we pass the target class as an additional argument to the attack. The attack is considered successful if and only if the model misclassifies the image as exactly the target class.

¹**Misclassified** is a key word here, which is interpreted differently in targeted and untargeted attacks. In untargeted attacks, any label other than the true one is considered a success; in targeted attacks, only the specified target label counts as a success. These details will be elaborated further in the attack-specific results.

To comprehend the mechanics of both types of FGSM attack, let us break down the key steps with mathematical expressions.

If we denote:

- the parameters of a model with θ ;
- the original input image with x ;
- the true label of the input image with y ;
- the label of a target class with y_{target} (only in the targeted setting);
- the loss function to train the neural network with $L(\theta, x, y)$,

Then we can define equations for both, targeted and untargeted attacks to generate adversarial examples.

In case of an untargeted attack we have:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y))$$

This **maximizes** the loss w.r.t. the true label y , pushing the prediction *away* from the correct class.

Whereas for the targeted attack we use:

$$x_{adv} = x - \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y_{target}))$$

This **minimizes** the loss w.r.t. a specific target class y_{target} , causing the original image to be classified *as that class*.

In both cases, ϵ is a small scalar controlling the perturbation magnitude, and $\text{sign}(\cdot)$ takes the sign of the gradient to apply the perturbation uniformly across each pixel direction.

Since FGSM requires access to model gradients, at its core this approach is a **white-box** attack. As a result of this attack, we get an image that is often almost indistinguishable from the original to the human eye, yet it deceives the image classification models. For example, Figure 2.1 below illustrates this, showing an original image and its adversarial counterpart that successfully fooled the victim model in an untargeted setting.



Figure 2.1 Original image with the true label "Barometer" (left), and the corresponding adversarial image that fooled model ViT-B/16 into classifying it as "Magnetic Compass" (right). The attack was conducted with the epsilon value $\epsilon = 0.2$.

In the following sections we present the results of FGSM experiments in both untargeted and targeted settings. In targeted attacks, to standardize the evaluation process, we test on 5 randomly chosen target classes: *Maltese Dog*, *Bee*, *Cassette*, *Hockey Puck* and *Pretzel*.²

Additionally, to demonstrate how the choice of ϵ affects the attacks, we tested the FGSM method with the following values:

- $\epsilon = 0.05$: minor perturbations, which are essentially not noticeable by human eye;
- $\epsilon = 0.2$: stronger perturbations, that are more noticeable, but still visually quite similar to the original.

Untargeted FGSM

We begin by evaluating FGSM in the untargeted setting, using the two fixed perturbation magnitudes: $\epsilon = 0.05$ and $\epsilon = 0.2$. Table 2.1 presents the attack success rates for each model.

Table 2.1 Attack success rates of untargeted FGSM attack on ViT and CNN models, using two different values of epsilon.

ϵ	ViT-B/16	ViT-L/32	ResNet50	ResNet152
0.05	68.09	68.28	56.36	51.67
0.2	76.36	79.18	53.91	51.74

It is interesting that FGSM achieves higher success rates against Vision Transformers than against CNNs for both values of epsilon, despite the fact that FGSM was originally developed for CNN-based models. From our experiments we can conclude that ViTs may be more vulnerable to this type of gradient-based attack, especially in the untargeted setting.

Additionally, we see that the attack does not benefit much from increasing the epsilon value from 0.05 to 0.2. This only showed a modest improvement, especially for CNN-based models, where the increase is negligible.

Finally, the attack conducted on our models shows comparable effectiveness against smaller and deeper models (within both architecture families). This implies that model depth alone does not guarantee robustness against untargeted FGSM attack.

Targeted FGSM

To evaluate the effectiveness of the FGSM technique under the targeted attack setting, we present two tables below. Tables 2.2 and 2.3 show the ASRs for each model across five target classes, using $\epsilon = 0.05$ and $\epsilon = 0.2$, respectively.

Despite the reasonable performance of FGSM in the untargeted setting, we see in these tables that the attack is essentially almost always unsuccessful in the targeted case. For $\epsilon = 0.05$, FGSM achieved attack success rates below 4% across all five target classes and models.

²These classes were randomly chosen at the beginning of the experiments, and are reused throughout the thesis for consistency.

Table 2.2 Attack success rates of the FGSM attack on ViT and CNN models in the targeted setting, evaluated across five target classes and for the epsilon value $\epsilon = 0.05$.

Target Class	ViT-B/16	ViT-L/32	ResNet50	ResNet152
Maltese Dog	1.91	1.54	1.11	0.80
Bee	1.20	1.35	1.14	0.42
Cassette	2.61	2.39	2.06	0.86
Hockey Puck	1.85	1.47	1.67	0.65
Pretzel	3.22	2.40	2.33	1.16

Table 2.3 Attack success rates of the FGSM attack on ViT and CNN models in the targeted setting, evaluated across five target classes and for the epsilon value $\epsilon = 0.2$.

Target Class	ViT-B/16	ViT-L/32	ResNet50	ResNet152
Maltese Dog	1.95	2.16	0.58	0.65
Bee	1.58	2.14	0.83	0.37
Cassette	3.26	3.28	0.70	0.44
Hockey Puck	2.26	2.49	0.76	0.43
Pretzel	3.93	4.02	1.05	0.70

Surprisingly, increasing the perturbation strength to $\epsilon = 0.2$ did not help with the attack effectiveness; it actually decreased the success rates in most cases to below **2%**. Therefore, larger perturbations do not necessarily improve FGSM’s performance in the targeted setting. This suggests that by increasing the perturbation magnitude, the attack might overshoot or just introduce noise.

To illustrate this, we present some of the perturbed images, with their corresponding original counterparts in the figure below:



Figure 2.2 Original images (left), and their corresponding adversarial images that failed to mislead victim model ResNet50 (right). The attack was conducted with the epsilon value $\epsilon = 0.2$, for a target class "Cassette".

These results highlight an important limitation of FGSM: as a single-step attack, it does not have the iterative nature needed to "move the image" toward the desired class. While it achieves moderate attack success rates in the untargeted setting, its simplicity limits the ability to craft precise perturbations for targeted attacks.

Therefore, targeted FGSM is considered weak, and more advanced methods like PGD are generally preferred for such scenarios.

2.3 Projected Gradient Descent (PGD)

Projected Gradient Descent (Madry et al., 2019) is also a gradient-based method, that is sort of an extension of FGSM. Unlike FGSM, PGD *iteratively* applies small perturbations to the input image. At each step, the perturbation is, again, computed using the gradient of the loss function, while respecting the predefined constraints and the range of change to ensure the imperceptibility of the attack. Due to this iterative nature, we expect to achieve better results with PGD, compared to those reported in the previous section for FGSM attack.

We conduct our experiments in both, targeted and untargeted settings. Therefore, we define the formulas of PGD method for both frameworks.

For the untargeted setting the formula can be defined in the following way:

$$x^{t+1} = \prod_{x+S} x^t + \alpha \cdot \text{sign}(\nabla_x L(\theta, x, y))$$

Whereas for targeted attacks we have:

$$x^{t+1} = \prod_{x+S} x^t - \alpha \cdot \text{sign}(\nabla_x L(\theta, x, y_{\text{target}}))$$

The gradient direction is, again, reversed for targeted attacks (just like in FGSM), to push the model *toward* predicting a specific target class y_{target} . Additionally:

- x_t is the adversarial example at step t ;
- α is the step size;
- $\text{sign}(\nabla_x L(\theta, x, y))$ is the sign of the gradient of the loss function L with respect to x ;
- $\prod_{x+S}$ is the projection operator that ensures that the updated adversarial example stays within a **valid perturbation set S**.

PGD is considered a strong first-order adversarial attack and is widely used for evaluating model robustness in the **white-box** setting. In the following sections we present the results of the experiments in two frameworks (untargeted and targeted) with the same target classes and epsilon values.

Untargeted PGD

We begin our PGD experiments with untargeted attacks, again using two fixed perturbation magnitudes. Table 2.4 below presents the attack success rates for each model.

Table 2.4 Attack success rates of untargeted PGD attack on ViT and CNN models, using two different values of epsilon.

ϵ	ViT-B/16	ViT-L/32	ResNet50	ResNet152
0.05	97.92	95.68	98.64	93.30
0.2	99.62	98.57	99.63	95.70

As we see from the table above, the effectiveness of PGD is significantly higher compared to FGSM. Across both values of ϵ , PGD consistently achieves ASRs above **93%** on all models, including both CNNs and Vision Transformers. This once again demonstrates that PGD is a strong adversarial method.

Just like for FGSM, the difference in attack success between small and large models within each architecture family is minimal, and this once again emphasizes that the depth of the model alone does not offer defense against gradient-based methods.

As for the difference between the epsilon values – again, we do not see any significant increase in attack success rates, only about 2-3% for every model. This suggests that the iterative nature of PGD is enough to create strong perturbations even at lower epsilon levels.

Targeted PGD

We now evaluate PGD in the targeted setting, using the same five target classes and perturbation magnitudes as before. Tables 2.5 and 2.6 show the ASRs for each model when the goal is to force the model to misclassify the image into a specific target class.

Table 2.5 Attack success rates of the PGD attack on ViT and CNN models in the targeted setting, evaluated across five target classes and for the epsilon value $\epsilon = 0.05$.

Target Class	ViT-B/16	ViT-L/32	ResNet50	ResNet152
Maltese Dog	95.82	79.56	91.75	72.97
Bee	89.58	73.69	95.80	66.86
Cassette	96.02	84.50	97.47	83.47
Hockey Puck	93.84	87.86	95.02	82.27
Pretzel	91.97	83.49	95.16	79.27

Table 2.6 Attack success rates of the PGD attack on ViT and CNN models in the targeted setting, evaluated across five target classes and for the epsilon value $\epsilon = 0.2$.

Target Class	ViT-B/16	ViT-L/32	ResNet50	ResNet152
Maltese Dog	98.87	94.38	96.36	81.63
Bee	96.86	92.78	98.33	75.88
Cassette	99.42	97.92	99.37	92.91
Hockey Puck	98.49	98.09	98.37	91.86
Pretzel	98.03	96.35	97.03	87.56

As expected, we see high success rates across all target classes and victim models under the targeted setting. In contrast to FGSM, the targeted PGD attack is much more effective. For $\epsilon = 0.05$, the attack success rates range from **~66%** to **~97%**, while increasing the perturbation strength to $\epsilon = 0.2$ boosts ASRs further to between **~75%** and **~99%**.

We present the Figure 2.3 below that demonstrates what adversarial images look like after applying the PGD attack. These images show perturbations achieved with both ϵ values, and they both managed to deceive the victim models.



Figure 2.3 Original images (left), and their corresponding adversarial images that fooled the victim model ViT-B/32 into misclassifying the images as "Casette" (left) and "Bee"(right). The attack was conducted with both epsilon values, $\epsilon = 0.05$ (image "Church"), and $\epsilon = 0.2$ (image "Dog").

Interestingly, perturbations are hardly visible, and even more unnoticeable than those in FGSM attacks.

To better visualize the effectiveness of the PGD attack as a whole, in both targeted and untargeted settings, we also include Figure 2.4 below that summarizes ASR values for all the previous experiments.

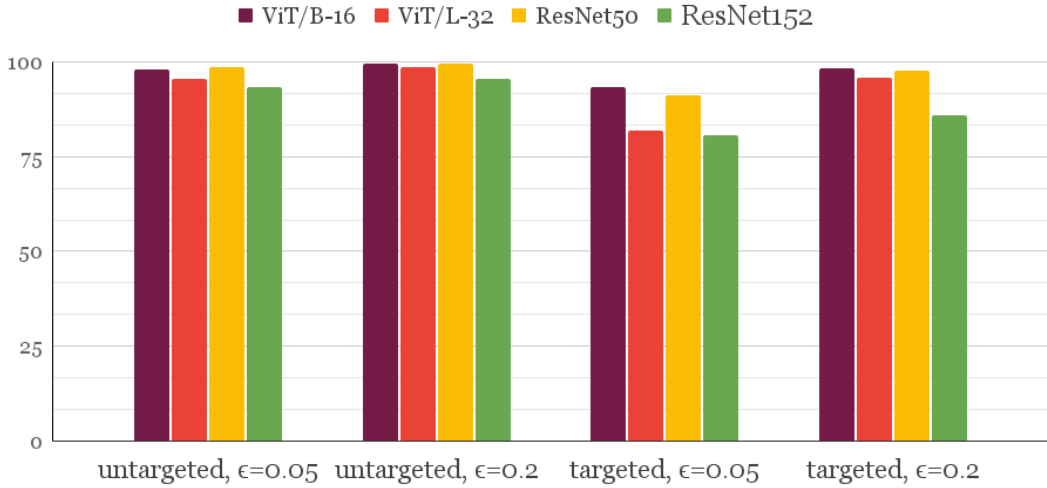


Figure 2.4 Attack success rates in targeted and untargeted settings, for both epsilon values. The ASR values for targeted attacks are averaged over all five target classes.

This figure emphasized one more trend – untargeted PGD is successful against every selected model, regardless of the architecture or depth. However, when it comes to the targeted approach, we see that deeper models in both architecture families are more robust. This is especially clear when the perturbation magnitude is lower ($\epsilon = 0.05$). The consistent improvement in ASR with a larger ϵ also demonstrates that PGD can better utilize the extra perturbation budget without destabilizing the attack.

These results once again confirm that PGD is a much more reliable method for generating both targeted and untargeted adversarial examples, especially when compared to single-step approaches like FGSM.

3 Random Position Patch Attack

In this chapter, we describe a patch-based adversarial attack that places an adversarial patch at a given position in the image, rather than perturbing individual pixels. Given the very specific, inherently patch-based structure of Vision Transformers, it also seems more natural to utilize this type of attack when generating adversarial examples. One such implementation is described in the paper "Designing Physical-World Universal Attacks on Vision Transformers" (Shao, 2024) ¹. In what follows, we analyze the paper in detail, implement the described attack from scratch, and build on it with our own modifications.

3.1 Overview of the Paper

The paper introduces a novel way of crafting adversarial examples specifically for ViT models. It does not target patches of the input images per se, instead, it relies on the randomness of the adversarial patch's position in the input image.

This approach is rather interesting and, as the author of the paper mentions, it represents a novel method of perturbing images. Unlike traditional methods, it does not rely on exploiting critical points of the model architecture. Therefore, the patch placement does not need to align precisely with the Vision Transformer "grid." Overcoming this constraint not only simplifies the patch generation and training pipeline, but it also enables deploying adversarial examples in the physical world, where precise placement cannot always be ensured.

The paper's approach essentially leverages a GAN-like architecture, where a sub-network, the **Generator**, is trained to create adversarial patches from random inputs. This is followed by a **Deployer**, which simulates random placement of the adversarial patch. The outputs of the deployer are then fed into the **Discriminator** - a victim model, that remains untouched and is only used to compute the adversarial loss, to optimize the generator. This architecture is visualized in the figure 3.1 below.

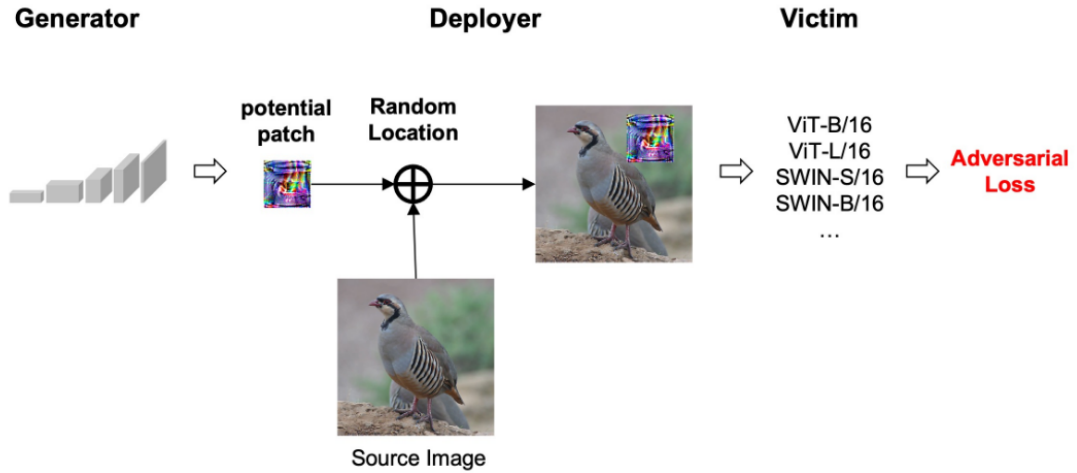


Figure 3.1 Overview of the G-Patch generating model. Source: Shao, 2024.

¹originally published as "Random Position Patch Attack"

3.2 Our Implementation

As a first step in our implementation of this attack, we tried to replicate the approach described in the original paper, although it is important to note that we incorporated additional adjustments to the pipeline to further explore the robustness of the adversarial models.

The implementation focused on recreating the key components: the generator and its training process, the deployment mechanism and discriminator logic.

3.2.1 Generator

The Generator serves as the backbone of the adversarial patch creation process. We start with random noise vectors, which are used to generate a possible patch through the convolutional layers of the generator network. As described in the original paper, our implementation uses 5 convolutional layers to progressively upscale the input noise to the desired patch size. Each layer, except the last one, is followed by batch normalization and ReLU activation functions. The final layer uses a Tanh-based activation function defined as:

$$Th(x) = k \cdot \tanh(x) + k$$

where $0 < k \leq 0.5$ to constrain the patch values within a normalized range. By the default setup, we use $k = 0.5$ to normalize the patch range to $[0, 1]$.

The generator architecture is constructed dynamically based on the specified patch size (64 or 80). Each layer applies a transposed convolution, followed by batch normalization and ReLU activation. Below is a simplified snippet of the layer construction:

```
layers.append(nn.ConvTranspose2d(input_dim, 1024, kernel_size=4, ...))
layers.append(nn.BatchNorm2d(1024))
layers.append(nn.ReLU(inplace=False))
```

Finally, the generator applies a thresholding function to ensure the patch values remain within the normalized range.

3.2.2 Deployment

The deployer mechanism is another essential part of this attack, and also where the main novelty of the approach lies. The author of the paper puts a strong emphasis on the fact that the deployer overcomes positional alignment constraints that most traditional patch-based attacks have. Therefore, we also ensured to implement a mechanism that places the generated patch at a random location on the input image. However, we slightly modified the technique. In particular, we only use "active" parts of the adversarial patches, instead of deploying the whole patch. This is because patches do not always occupy every pixel – some regions remain black and do not contribute to the perturbation. We treat the black pixels as *transparent* (for example, see figure 3.5). This way, we essentially apply only the *active* parts of the patch, and also minimize the size of the modified region of the original image.

To be more precise, first we generate a binary mask M with the same spatial dimensions as the input image, where pixels within the randomly chosen patch

region are set to 1, and all others are set to 0. Then, we apply the previously generated patch p within the mask region (where pixels are set to 1) and get the final mask M_p . The modified image is then generated from the initial image x by the deploy function $T(M_p, x)$:

$$T(M_p, x) = M_p + (1 - M_p) \cdot x$$

For more clarity, we include a pseudocode of the deployer mechanism:

Algorithm 1 Patch Deployment

Require: Input image: x , adversarial patch: p

```

1:  $M \leftarrow \text{Zeros}(x)$  ▷ binary mask with same shape as  $x$ 
2:  $k \leftarrow \text{Randint}(0, \text{shape}(x)[0] - \text{shape}(p)[0])$ 
3:  $l \leftarrow \text{Randint}(0, \text{shape}(x)[1] - \text{shape}(p)[1])$  ▷ random position ( $k, l$ ) within  $M$ 
4: for  $i \leftarrow 0$  to  $\text{shape}(p)[0] - 1$  do
5:   for  $j \leftarrow 0$  to  $\text{shape}(p)[1] - 1$  do
6:      $M[k + i, l + j] \leftarrow 1$ 
7:   end for
8: end for
9:  $M_p \leftarrow M \cdot p$  ▷ place the patch within the mask
10:  $\tilde{x} \leftarrow M_p + (1 - M) \cdot x$  ▷ deploy patch onto image
11: return  $\tilde{x}$  ▷ final adversarial image

```

So, essentially, the deployer replaces a random part of the initial input image, with an adversarial patch, also called a **G-patch** (see Fig. 3.1), which does not necessarily align with the token grid of the ViT model architecture.

This approach is particularly interesting and useful in real-world applications. Of course, it is difficult to place the adversarial patches with pixel precision in real-world scenarios, as environmental factors can significantly affect placement. By ensuring that the patch is randomly positioned during training, this method aims to increase robustness and generalizability. In addition, because the deployer does not rely on strict spatial constraints, it can be adapted to different architectures beyond Vision Transformers, broadening its potential impact.

3.2.3 Discriminators

In the original paper, the discriminator in the network is composed of a single victim model (for example, ViT-B/16, SWIN-B/16, DeiT-S/16, etc.). As mentioned earlier, in addition to original models, we also experimented with several CNN models, such as ResNet-50, ResNet-152, and VGG-16-BN. This offers an interesting insight into how different architectures respond to adversarial perturbations and highlights the relative robustness of CNNs compared to ViTs in specific scenarios.

To further explore the robustness and generalizability of the generators, we extended the original implementation by incorporating an ensemble of models during training. On one hand, this allows us to investigate how the generator adapts when trained on multiple ViT models and whether the learned perturbations transfer to other ViT victim models. On the other hand, we also aim to mix ViTs and CNNs within the ensemble to analyze the effects of training across different architectures.

3.2.4 Adversarial Loss

It is important to note that unlike traditional GANs, where the discriminator and generator are trained adversarially in a sort of "competitive" manner, the discriminator in our network remains unaltered throughout the training process. This allows the generator to adapt to a static victim model without modifying the way it classifies inputs. By doing so, we ensure that the generated adversarial patches remain effective against an unmodified real-world model, which, in turn, makes them more practical for deployment.

We only rely on the loss function, which, in case of a single discriminator is defined as:

$$L = \log(\text{softmax}(Pr(\hat{y}|T(M_p, x))))$$

where the $\hat{y}$ is the target class and $\hat{y} \neq y$, Pr is the prediction of the discriminator with respect to class $\hat{y}$.

In the case of multiple discriminators, we use a weighted loss function:

$$L_{weighted} = \frac{\sum_i L(x_i)}{n}$$

where n is the number of discriminators in the network. This ensures that the generator does not overly specialize in attacking a single model but instead learns perturbations that generalize across different architectures. This is particularly useful for generating adversarial patches that can transfer between models, increasing their effectiveness in black-box attack scenarios.

3.2.5 Training Process

Training the generator involves minimizing an adversarial loss that measures the effectiveness of the generated patch against the victim model. The patch is optimized to "force" the model to classify images as a predefined target class, regardless of the original label.

In the original implementation, the author uses the ImageNet1K dataset to train the generator, and optimizes classification loss L using the Adam optimizer ($lr = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$). The training for each configuration is conducted over 80 epochs, selecting the patch that achieves the best performance as the final adversarial patch. Additionally, input images are standardized to dimensions of 224×224 , with pixel values normalized to the range of $[0,1]$ for consistency.

To assess the attack success rate (ASR) of a patch, the paper uses the following formula:

$$ASR = \frac{Q}{P}$$

where Q denotes the number of images that were misclassified by victim models as the target class after deploying the adversarial patch, and P denotes the total number of images.

We keep most of the basic configuration of the network setup to replicate the results of the original paper as closely as possible. However, we conduct the experiments over 40 epochs instead of 80, since our results (as we will discuss in the experimental results section) show that the attack success rate converges after approximately 30 epochs (usually even earlier). Another difference in the

configuration worth mentioning is the dataset – instead of ImageNet1K, we use ImageNetV2 (Recht et al., 2019), which contains 10 images for each of the 1000 classes.

Furthermore, the training process for each experiment is done over 5 repetitions – since this ensures reliable performance evaluation, regardless of the impact of randomness, like bad initial settings, that could make the network take an unlucky training path early on.

To make the results even more consistent, we generate and fix 5 random noise vectors, which are later used to evaluate the saved generators.

The actual training process proceeds as follows: for each epoch, the data is loaded batch by batch, and for each image in the batch, we generate a random noise vector that is then fed into the Generator to create adversarial patches. These patches are deployed on batch images at random positions using the deployer mechanism and then we pass the modified images to the victim model (or ensemble of models). The loss is computed based on the model’s predictions and the target class, and then it is backpropagated to optimize the generator. This way, we iteratively refine and upsample adversarial patches to obtain higher attack success rates.

Our implementation saves the generator at the end of each epoch and proceeds to evaluation only after the training is fully complete.

3.2.6 Evaluation

As mentioned earlier, the evaluation process uses a fixed set of random noise vectors. These vectors are predefined at the start and remain consistent across all evaluations to ensure fair comparison between different generator checkpoints. For each training iteration, we evaluate generators saved at every epoch using these same five noise vectors as input. This allows us to monitor how well each generator performs over time under identical conditions.

To determine the best-performing generator, we calculate the average ASR across the five noise vectors for each checkpoint. The generator with the highest average ASR is then selected as the most effective one. Using the same five noise vectors across all epochs and iterations ensures that the comparison is fair and consistent – any improvement in performance can be attributed to the generator’s progress, rather than differences in the input.

Furthermore, by averaging ASR across multiple noise vectors, we reduce the impact of any individual vector that might happen to work unusually well (or poorly) by chance. This makes it more likely that the selected generator performs well across a broader range of inputs, rather than overfitting to a single lucky vector. In other words, we select the generator that generalizes best across the space of possible noise vectors – at least, as approximated by our fixed set of five.

Once selected, this generator, along with the corresponding adversarial patches (one for each random noise vector input), is saved for later use in testing attacks against different target models.

In the following sections of this chapter we discuss the results yielded by this setup.

3.3 Experiment Results

To evaluate the effectiveness of our adversarial patch generation approach, we conducted a series of experiments targeting selected image classes. Our experiments focused on the following five target classes: *Maltese Dog*, *Bee*, *Cassette*, *Hockey Puck*, and *Pretzel*. These classes were selected randomly, at the start and remained fixed throughout all experiments. Since they were chosen from the same dataset used for training, we excluded them from both the training and evaluation phases of the generator models to ensure that the model was never exposed to its target classes in advance.

As mentioned above, we reduced the number of training epochs to 40, as our primary evaluation metric – the Attack Success Rate – consistently improves throughout training and tends to converge in the later epochs. This trend is illustrated in the two figures below, which show how both, the average loss and ASR of a 64×64 size adversarial patch evolve over the course of 40 epochs.

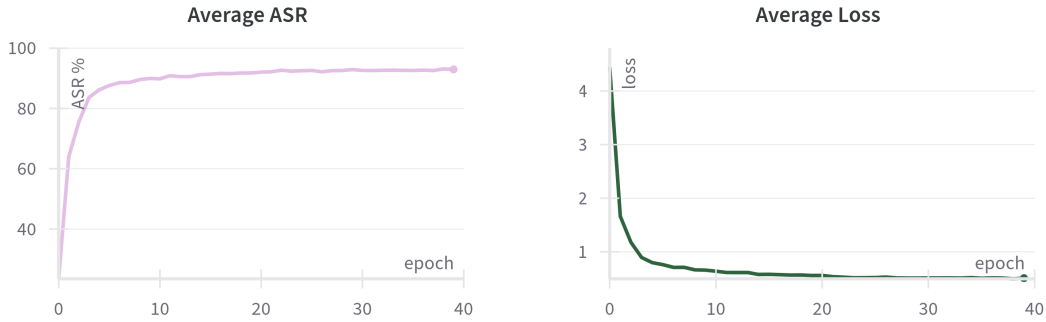


Figure 3.2 Training metrics of a generator creating 64×64 adversarial patches. The discriminator model is ViT/B-16; target class is "Maltese Dog".

To illustrate the results of our experiments in a more consistent and clear manner, we separate the findings for 64×64 and 80×80 patches. In the following sections we provide tables, charts, and patch images for each patch size and configuration setting.

3.3.1 64×64 Patch Results

We begin with a visual summary of the generator’s performance over the epochs. Figure 3.3 provides a comparison of the initial random noise, followed by adversarial patches of target class *Bee* at epoch 1 and the best-performing epoch.

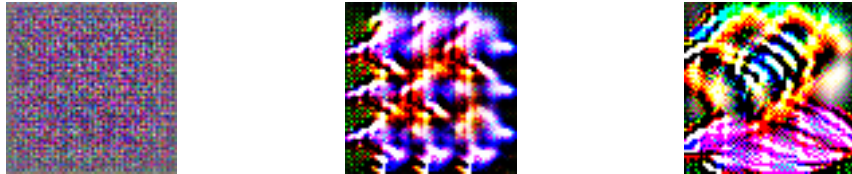


Figure 3.3 Target class "Bee"; Victim Model ViT/B-16. Initial random noise vector (left); adversarial patch after the first epoch (middle) and the final, best-performing epoch (right).

This visual comparison highlights how the patch evolves from an unstructured, noisy pattern to a highly refined and vibrant design. By the final epoch, the patch even resembles the features of an actual bee. This transformation nicely reflects the generator’s ability to optimize the patch for maximum attack success.

Figure 3.4 further demonstrates the generator’s adaptability by showing the best-performing patches for the *Bee* target class across six additional discriminator models. Despite all targeting the same class, the patches exhibit notably different visual characteristics. This illustrates how each model’s internal feature representations and vulnerabilities are uniquely exploited by the generator.



Figure 3.4 Best-performing adversarial patches for the target class "Bee" across six different victim models: ViT-B/32, ViT-L/16, Swin-B, ResNet50, ResNet152, and VGG16-BN (from left to right, and top to bottom). Each patch is the result of a separate generator trained to attack the respective model.

Even though these patches might seem large, it is important to note that they only occupy around 8% of the original image. To better visualize this, we present some of the modified images (at Fig. 3.5) that were misclassified by the victim models.



Figure 3.5 Perturbed images that were misclassified by the victim model ResNet152 as the target class *Bee*. Patch generator was trained on the same discriminator model. Original image labels were: "Elephant", "Tree Frog" from left to right, respectively. This figure also demonstrates the mild difference between the patches from different runs of the same experiment.

Finally, we turn to the details of the training process across all 7 victim models and 5 target classes. Table 3.1 presents the attack success rate scores obtained by each generator over 5 runs per target class. For compactness, the target models are abbreviated (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50, etc.).

Table 3.1 ASR scores, evaluated over 5 runs per target class. Target models are abbreviated for compactness (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50).

TC	It. #	VB/16	VB/32	VL/16	Sw-B	RN50	RN152	VGG
Maltese Dog	1	92.99%	62.69%	42.88%	96.09%	86.17%	90.38%	97.42%
	2	92.86%	62.80%	43.47%	95.03%	84.85%	87.40%	97.22%
	3	92.48%	62.45%	37.62%	92.04%	79.64%	91.98%	96.97%
	4	92.86%	62.77%	45.17%	90.27%	91.25%	91.00%	97.02%
	5	92.36%	63.57%	47.28%	96.27%	78.18%	90.41%	97.44%
Bee	1	84.33%	40.58%	47.49%	96.51%	50.01%	78.89%	94.35%
	2	84.94%	23.70%	50.21%	95.61%	59.33%	69.32%	93.02%
	3	82.49%	37.23%	47.82%	95.87%	59.67%	78.84%	93.63%
	4	81.63%	27.53%	46.34%	95.87%	56.37%	54.70%	94.29%
	5	84.15%	39.66%	40.96%	95.98%	59.93%	83.95%	94.17%
Cassette	1	92.07%	31.33%	58.65%	96.57%	69.59%	79.41%	96.39%
	2	92.39%	33.15%	48.94%	96.54%	78.40%	79.48%	96.49%
	3	91.58%	35.46%	61.14%	95.39%	76.02%	77.59%	96.27%
	4	92.42%	31.94%	43.50%	95.88%	64.92%	79.41%	96.43%
	5	92.75%	38.00%	55.75%	95.87%	72.69%	76.40%	96.48%
Hockey Puck	1	82.11%	36.17%	36.22%	92.56%	42.91%	48.30%	88.88%
	2	82.95%	36.90%	37.32%	92.27%	61.93%	29.78%	92.29%
	3	80.18%	36.51%	38.05%	92.47%	48.49%	28.17%	90.10%
	4	80.29%	36.55%	28.64%	91.54%	69.09%	39.05%	91.01%
	5	80.03%	36.88%	36.57%	92.09%	45.22%	28.04%	89.45%
Pretzel	1	90.11%	71.62%	65.22%	95.79%	89.66%	80.79%	96.90%
	2	90.06%	71.28%	73.48%	96.21%	86.60%	96.00%	96.93%
	3	89.25%	72.10%	70.15%	95.73%	88.42%	78.52%	96.97%
	4	90.16%	71.35%	73.15%	96.16%	44.55%	79.69%	97.04%
	5	89.87%	72.75%	73.72%	96.54%	83.41%	76.22%	96.87%

To better capture overall trends, we also include a visual summary of the table in Figure 3.6, which shows the average ASR values (across all iterations) for each target class and model. This figure provides insight into the robustness of different architectures against G-patch attacks.

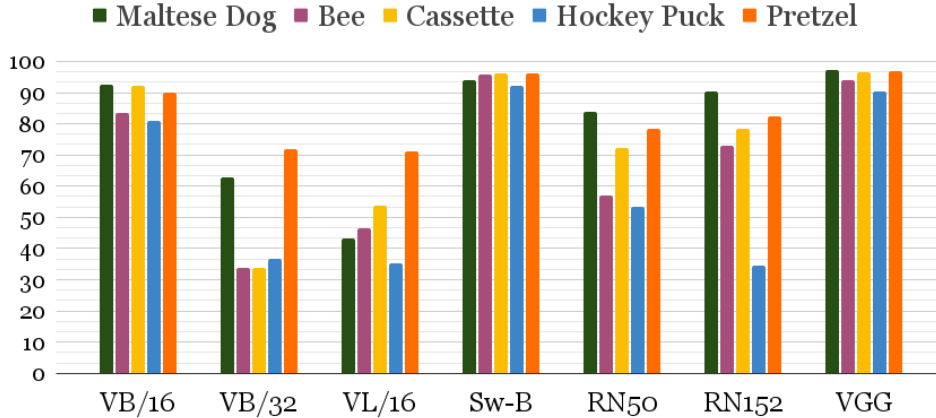


Figure 3.6 Average ASR per target class and model. Shows model vulnerability to G-patch attacks across all classes.

It is quite easy to spot the least robust models: Swin-B and VGG16-BN are highly susceptible to our patches, with higher than **90%** ASR for all target classes. ViT/B-16 also appears relatively easy to fool, with ASR values consistently above **80%** across the board.

On the other hand, ViT/B-32 and ViT/L-16 show more robustness. Their ASR values are noticeably lower, especially for more challenging target classes such as *Bee*, *Cassette*, and *Hockey Puck*. This suggests that larger Vision Transformers may have better inherent defenses against our patch-based attacks.

In CNN-based models, ResNet-50 and ResNet-152 show mixed results. ResNet-50 demonstrates high ASR for some classes – such as *Maltese Dog* and *Pretzel* – but lower ASR (around **50–60%**) for others like *Bee* and *Hockey Puck*. Interestingly, despite being a deeper model, ResNet-152 exhibits similar resistance levels – with a noticeable improvement only for *Hockey Puck*. This might suggest that deeper architectures, particularly in Convolutional Neural Networks, do not necessarily guarantee better robustness against G-patch attacks.

Another interesting observation is the variance in ASR scores across different runs for the **same** target class and model. For instance, ResNet-152 shows quite a big variance when we train the generator against it, for the target class *Bee*: from **54.7%** to **83.95%** ASR across five runs (see Appendix for corresponding patches). This seems to be a trend in models that we previously identified as more resistant. However, easy-to-fool models like Swin-B, VGG16-BN and ViT/B-16 show far more consistent ASR values across the runs. This, once again, confirms our earlier observation: once a model is known to be vulnerable, nearly any generator initialization is likely to lead to high ASR.

Finally, we note that some target classes produce more generalizable adversarial patches than others. Classes like *Pretzel* and *Maltese Dog* obtain consistently high ASR across models, while other classes, such as *Hockey Puck* or *Bee* show variance that is more model-specific. If we disregard the highly susceptible models, these differences between target classes become even more noticeable, as visualized in Figure 3.7

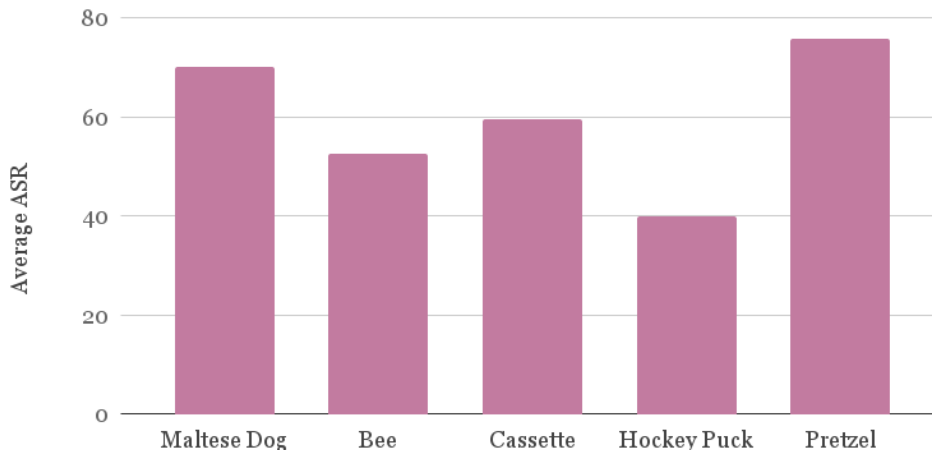


Figure 3.7 Average ASR per target class (robust models only). Highlights class-wise patch generalizability, excluding easiest-to-fool models.

3.3.2 80×80 Patch Results

We repeat the above analysis using 80×80 pixel patches to examine whether increasing patch size leads to improved attack success or consistency.

Let us start by showing some of the most effective patches for a target class *Maltese Dog*, trained individually for different models. These are visualized in Figure 3.8.



Figure 3.8 Best-performing adversarial patches for the target class "Maltese Dog" across six different victim models: ViT-B/16, ViT-B/32 and ViT-L/16 (top row from left to right); and ResNet50, ResNet152, VGG16-BN (bottom row from left to right).

We also present four additional patches generated from different training runs for the ViT-B/32 model. This illustrates how the best-performing patches can vary across different iterations. These are shown in Figure 3.9.

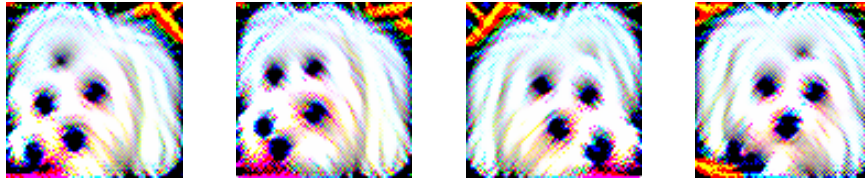


Figure 3.9 Best-performing adversarial patches for the target class "Maltese Dog" across 4 iterations for the victim model ViT-B/32.

It is interesting how the patches generated for the Vision Transformer models look quite consistent – they all resemble a single Maltese dog. This suggests that the generators trained on ViTs tend to focus on creating one clear, coherent concept. On the other hand, the patches for the CNNs seem to have multiple smaller dog-like faces or features in one patch, almost like a collage of mini dogs. This might mean that CNNs respond more to local features, while ViTs capture more of the overall shape or idea. Similar behaviour can also be observed in the 64×64 patches (see Fig. 3.4).

Interestingly, the patches generated on CNNs look quite similar to the images produced in the "Inceptionism: Going Deeper into Neural Networks" study (Mordvintsev et al., 2015). This work aimed to visualize how neural networks internally view and represent different concepts. This was done by generating random objects like "Banana" from random noise input. Like our patches trained on CNN models, these visualizations tend to contain multiple fragmented object parts across the image. Some of the examples are shown below in Figure 3.10.



Figure 3.10 Example outputs from the “Inceptionism” study (Mordvintsev et al., 2015), including visualizations for images *Starfish*, *Banana*, *Parachute*, and *Anemone Fish* (from left to right).

Returning to our analysis, to get a sense of how the 80×80 adversarial patches appear in practice, we present several examples of modified images in Figure 3.11.

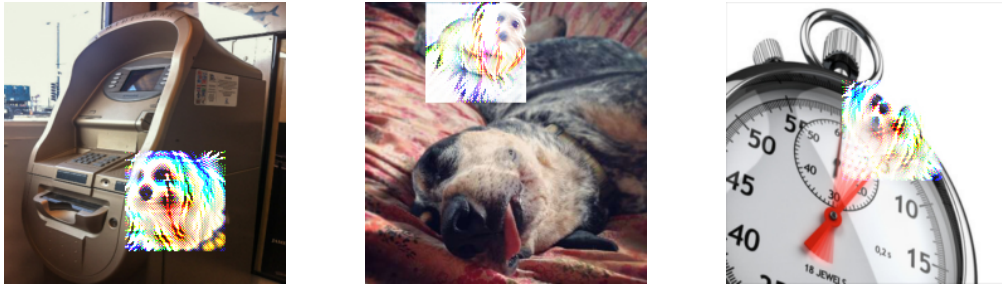


Figure 3.11 Example adversarial images that were misclassified by a victim model Swin-B. Actual labels of the images were “ATM”, “Bluetick Coonhound” and “Stop-watch” from left to right, respectively. The patch generator was trained on ViT-B/16.

Although these patches appear larger than 64×64 , they still only cover approximately **12%** of the original image. Despite this, we observe a considerable increase in attack success rates, especially on larger or previously more robust models.

Table 3.2 below summarizes the average attack success rates for each target class across all models. Unlike the previous table (3.1) for 64×64 patches, which listed the ASR from each of the five individual runs, this table presents *only the average* scores across the five runs for compactness.

Table 3.2 Average ASR for 80×80 patches across five runs per target class. Target models are abbreviated for compactness (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50, etc.).

Target Class	VB/16	VB/32	VL/16	Sw-B	RN50	RN152	VGG
Maltese Dog	96.14%	83.17%	76.50%	97.70%	94.04%	93.15%	98.55%
Bee	87.89%	63.02%	69.21%	97.88%	83.30%	85.45%	97.43%
Cassette	96.23%	72.63%	75.91%	98.07%	81.85%	92.39%	98.39%
Hockey Puck	91.15%	63.01%	70.97%	96.84%	83.17%	71.44%	95.76%
Pretzel	95.52%	84.95%	83.26%	97.83%	88.74%	94.06%	98.48%

The results are also visualized in Figure 3.12, which summarizes the table of averaged ASR scores and helps identify trends more clearly.

Both figures show us that all models exhibit increased ASR values compared to the 64×64 setup. This is expected, as a larger patch not only allows the generator

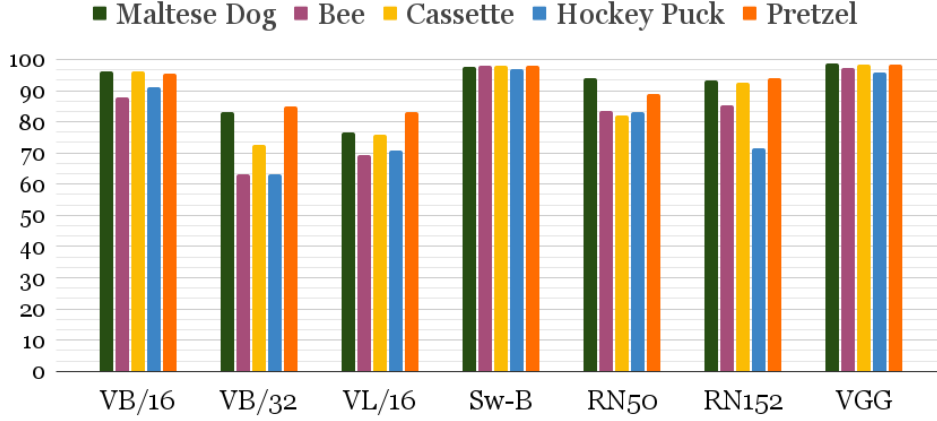


Figure 3.12 Average ASR per target class and model. Shows model vulnerability to 80×80 sized G-patch attacks across all classes.

to exploit the victim models more and encode more "perturbation capacity", but it also simply occupies a bigger region of the image, increasing the likelihood of covering something crucial for the victim model to correctly classify the image.

While Swin-B and VGG16-BN were already very vulnerable to 64×64 adversarial patches, we still see a small increase in ASR scores for 80×80 patches. For these models bigger patches outperform the smaller ones across all target classes, with ASR values consistently above 96%.

For more resistant ViT models, particularly ViT-L/16 we see a jump from moderate robustness of 40-60% to 70-80% ASR. We see an improvement of around 15-35% in the ASR scores across all target classes against ViT-B/32.

CNNs, which previously showed mixed robustness, now reach ASR levels of 78-97% across all target classes. 80×80 size patches are quite successful even when trained on the more difficult target classes like *Bee* or *Hockey Puck*.

Below we present two figures that visualize on average how the ASR scores improved with 80×80 patches on ViT-L/16 and ResNet152.

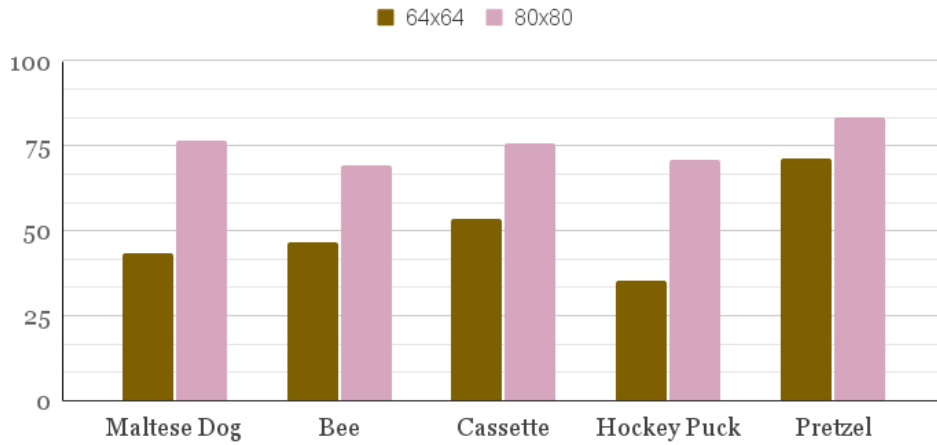


Figure 3.13 Average ASR values across all target classes for the victim model ViT-L/16. Brown bars represent ASR values of 64×64 patches, pink ones – of 80×80 .

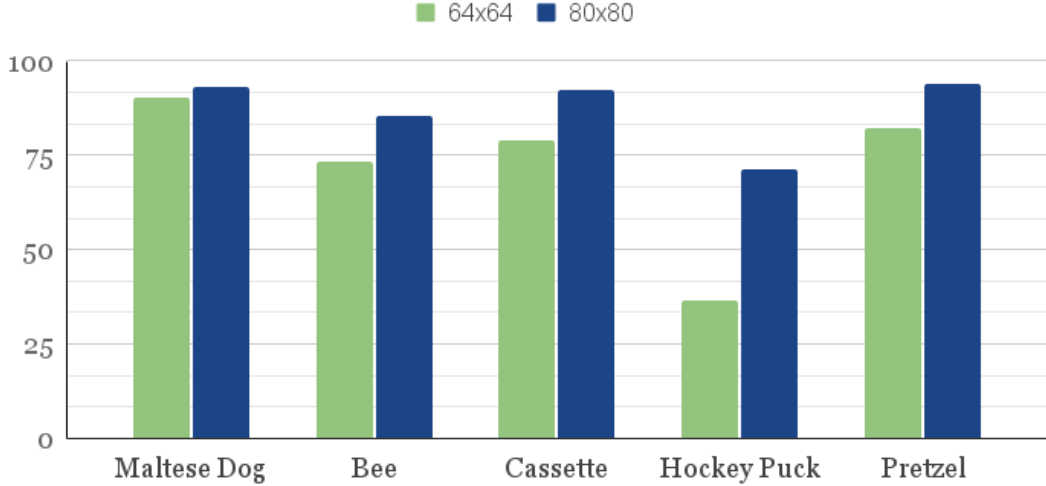


Figure 3.14 Average ASR values across all target classes for the victim model ResNet152. Green bars represent the ASR values achieved by 64×64 patches, blue ones – by 80×80 .

Overall, these results highlight a clear trend: increasing the patch size from 64×64 to 80×80 significantly improves the attack success rate across all model families and target classes. Just like shown in Figures 3.13 and 3.14 above, even models that were relatively robust at smaller patch sizes: ViT-L/16 and ResNet152, become considerably more vulnerable when attacked with larger patches.

With these results obtained, we now turn to a comparison between our results and those reported in the original paper.

3.3.3 Comparing to original paper

To compare our results with those presented in the original paper, we report both **class-wise ASRs** and the **average ASR across all five target classes**. This is because the original paper does not specify which target classes were used; instead, it reports averaged ASRs over synthetic tests.

In our experiments, we observed a significant variability across our target classes: certain classes like *Maltese Dog* and *Pretzel* consistently achieved higher ASRs, while classes like *Hockey Puck* and *Bee* were more challenging and resulted in lower attack success rates. Since we are averaging over a small number of classes, this variability has a big impact on the overall average ASR and causes a big decrease in its value.

Therefore, while we provide an average ASR for reference (see Tab. 3.3), we caution against using it as the only (or main) metric for comparison. We believe that looking at each target class separately gives a clearer picture, especially since we do not know which classes were used in the original paper. For this, we provide Tables 3.4 and 3.5 which present more detailed results for each of the five target classes using 64×64 and 80×80 patches, respectively.

As demonstrated in Table 3.3, we see that our results are quite similar to those in the original paper. In some cases, like with 64×64 patch attacks on ViT-B/16 and Swin-B, our implementation even slightly outperforms the original one.

Table 3.3 Average ASR across all five target classes, compared with the ASR values reported in the original paper.

Method	ViT-B/16	ViT-L/16	Swin-B
Our G-patch (64)	87.9%	50.0%	94.9%
Original (64)	75.4%	63.3%	92.8%
Our G-patch (80)	93.6%	75.2%	95.7%
Original (80)	96.1%	87.7%	98.3%

Table 3.4 ASR results for each target class using 64×64 adversarial patches on ViT models, compared with the ASR values reported in the original paper.

Models	Maltese Dog	Bee	Cassette	Hockey Puck	Pretzel	Or. Paper
ViT-B/16	92.7%	83.5%	92.2%	81.1%	89.9%	75.4%
ViT-L/16	43.4%	46.6%	53.6%	35.4%	71.1%	63.3%
Swin-B	93.9%	96.0%	96.1%	92.2%	96.1%	92.8%

Table 3.5 ASR results for each target class using 80×80 adversarial patches on ViT models, compared with the ASR values reported in the original paper.

Models	Maltese Dog	Bee	Cassette	Hockey Puck	Pretzel	Or. Paper
ViT-B/16	96.1%	87.9%	96.2%	92.2%	95.5%	96.1%
ViT-L/16	76.5%	69.2%	75.9%	71.0%	83.3%	87.7%
Swin-B	97.7%	83.3%	98.1%	96.8%	98.9%	98.3%

A noticeable difference is observed with ViT-L/16, where the original paper reports **7-10%** higher ASR values for both 64×64 and 80×80 patches. This difference could be due to the fact that we decreased the number of epochs from 80 to 40 – larger models like ViT-L/16 might require more training time to converge to an optimal generator.

In Tables 3.4 and 3.5 we see the difference more clearly. For example, in the 64×64 setting, our generators trained on ViT-B/16 and Swin-B models consistently outperform the original result across all five target classes. With ViT-L/16, however, we see high variability – since this is a deeper and more robust model, it shows a wide range of success rates, from **35.4%** (significantly lower than the original average) to **71.1%** (slightly higher).

It is also possible that additional differences in implementation, like patch initialization, optimization settings, or target class selection contributed to the discrepancy, especially for ViT-L/16. Without access to the exact settings and data used in the original work, a perfect reproduction is inherently difficult.

Overall, our results align well with the original ones, which once again confirms the effectiveness of this patch-based approach. While differences exist, they are explainable by changes in the configuration settings.

In the next section, we explore the transferability of our adversarial patches across models, evaluating whether a patch trained on one model (or an ensemble of models) can effectively attack another.

3.4 Transferability

In addition to replicating the results of the original implementation, we also aim to investigate the transferability of adversarial attacks. This serves as an important evaluation metric – assessing whether the generated patch can successfully transfer to different models provides valuable insight into the robustness and generalizability of the attack.

3.4.1 Intra-Family Transferability

Our analysis begins with exploring **intra-family** transferability – that is, how well adversarial patch generators trained on one model transfers to other models within the same architecture family. For example, we examine how a generator trained on ViT-B/16 performs on a larger model like ViT-L/16, or ViT-B/32.

Beyond single-model training, we also explore an ensembling strategy, where a generator is trained on multiple models from the same family and then tested on others within that group.

In this setting, we use ViT-B/16, ViT-B/32, ViT-L/16 and Swin-B models from Vision Transformers.

Single Model Transferability

We first present results where the generator is trained on a single ViT model, and then evaluated on the remaining ViT models. Given that we use two patch sizes (64×64 and 80×80), five target classes, and save the best-performing generator for each training run, we narrow our analysis in this section to two representative target classes *Bee* and *Pretzel*.

We choose these specific classes because the former, *Bee* is a relatively hard target to train the generator for, while the latter, *Pretzel* seems to be an easier one. Therefore, we think that the results even in this limited framework will still provide important insights and valuable information.

For each setting, we report:

- the average ASR across five training runs (i.e. five generators);
- the best ASR observed.

We do this since in some cases the average may differ from the best observed performance by quite a lot. Therefore, we aim to capture not only the generator’s typical performance, but also its potential under ideal conditions.

Tables 3.6 and 3.7 present these results on ViT models, for the two representative target classes, and patch sizes 64×64 and 80×80 , respectively. Diagonal entries of the tables represent ASRs when the generator is evaluated on the same model it was trained on; while off-diagonal cells show how well the attack transfers to other models within the same family.

These tables demonstrate some clear trends in the transferability experiments. We see that the generators trained on ViT-L/16 have the highest transferability rate, especially to ViT-B/16. This was expected, since ViT-L/16 is the deepest model and we would assume that if an attack manages to mislead it, it would likely mislead the model of the same architecture, only with less depth.

Table 3.6 Transferability results for 64×64 size patches. Each cell contains two values separated by a slash (avg/best): the average ASR across five runs (left) and the best ASR observed (right). Diagonal entries represent same-model evaluation; off-diagonal entries show transferability.

Train Model	Target	ViT-B/16	ViT-B/32	ViT-L/16	Swin-B
ViT-B/16	Bee	83.5/85	0.6/ 0.8	14.6/ 16.2	32.8/ 37.8
	Pretzel	90/90.2	20.1/ 24.3	32.5/ 33.9	46.8/ 48.4
ViT-B/32	Bee	4.7/ 7.3	33.7/40.6	1.9/ 2.8	12/ 22.8
	Pretzel	26.3/ 27.8	71.8/72.8	9.5/ 10.2	20.6/ 24.1
ViT-L/16	Bee	24.6/ 36.9	0.7/ 1.5	46.6/50.2	8.2/ 15
	Pretzel	73.8/ 76.9	10.1/ 16.9	71.1/73.7	24.8/ 32.2
Swin-B	Bee	0.1/ 0.1	0.1/ 0.1	0.1/ 0.1	96/96.3
	Pretzel	1.7/4	0.2/ 0.5	0.5/ 1.4	96.1/96.6

Table 3.7 Transferability results for 80×80 adversarial patches across ViT models. Format follows Table 3.6 above.

Train Model	Target	ViT-B/16	ViT-B/32	ViT-L/16	Swin-B
ViT-B/16	Bee	87.9/89.4	1.2/ 1.3	17.2/ 20.5	23.7/ 34.6
	Pretzel	95.5/95.9	18.9/ 33	48.4/ 59.6	57.2/ 68.1
ViT-B/32	Bee	17.6/ 24.1	63/69.4	6.2/ 8.8	16.7/ 23.7
	Pretzel	52.8/ 55.2	85/85.3	12.7/ 15.9	41.6/ 42.9
ViT-L/16	Bee	7.4/ 10.4	0.6/ 0.8	69.2/72.4	3.6/ 7
	Pretzel	80.6/ 84.9	23/ 46.9	83.3/85.3	37.4/ 50
Swin-B	Bee	0.1/ 0.2	0.1/ 0.2	0.2/ 0.2	97.9/98.1
	Pretzel	1/ 2.5	0.2/ 0.3	0.3/ 0.6	97.8/98.1

The generators trained on ViT-L/16 perform relatively well on Swin-B too, even though this is a completely different model with a distinct structure and internal architecture. This could be explained by the fact that Swin-B, in general, is a very susceptible model to adversarial perturbations.

ViT-B/32, another large model, though not as deep as ViT-L/16, shows the best transferability rate on the ViT-B/16 model.

ViT-B/16, which appears to be the most sensitive towards transferred attacks, has relatively strong transfer success rates itself. The generators trained on this model have the most consistent transferability performance across other ViTs.

Swin-B proved to be the weakest model against adversarial examples in previous experiments, and as the tables above show, it is also highly vulnerable to attacks transferred from other models. Additionally, generators trained on Swin-B are not good at generalizing and their transferability rates stay consistently below **2%**.

The tables also highlight the difference in the transferability of patches targeting different classes. Generators trained for the target class *Bee* struggle more to generalize and transfer, compared to those trained for the target class *Pretzel*. This is especially noticeable in the generators trained on the ViT-B/32 model.

In summary, transferability varies significantly across models and target classes. While some generators consistently generalize across other ViTs, others fail to generalize despite being highly vulnerable to transferred attacks. This highlights the limitations of single-model training and inspires us to explore ensemble-based approaches to improve transferability.

Ensembling

To further strengthen the attacks and improve their generalizability, in this section we explore the results of training on *ensembles*² of Vision Transformers. In particular, we train on an ensemble of all but one ViTs and test the patches on the remaining, unseen one. This is designed to encourage the generator to learn more generalizable features across models within the same family.

The results of this experiment for Vision Transformers are presented in Tables 3.8 and 3.9, for patch sizes 64×64 and 80×80 , respectively. We note that starting from this chapter, the training on ensembles will only be conducted once due to limited resources (since training takes around 30 hours). Therefore, the tables follow a slightly different format than those for single-model transferability.

In Tables 3.8 and 3.9, the first column lists the three-model ensemble used for training, while the other columns show ASR results on each individual model. The model not included in the ensemble (i.e., the test-time target) represents the main focus of our transferability analysis and is in **bold** in each row. The other values are included for completeness and to offer insight into the generator’s performance on *seen* models. Additionally, the last column contains the ASR results of the ensembles themselves (obtained during training).

Table 3.8 Transferability results of a generator trained on ensembles of three ViT models and tested on the remaining one, using 64×64 adversarial patches. Model names are shortened for compactness; and the last column, under "Ens. ASR", lists ASR results of the ensembles themselves.

Ensemble	Target	VB/16	VB/32	VL/16	Sw-B	Ens. ASR
VB/32, VL/16, Sw-B	Bee	27.8	21.9	27.1	77.7	<i>43.9</i>
	Pretzel	73.1	48.8	55.8	84.2	<i>68.9</i>
VB/16, VL/16, Sw-B	Bee	72.3	0.8	31.2	83	<i>71.1</i>
	Pretzel	86.1	0.7	58.3	83.1	<i>83.5</i>
VB/16, VB/32, Sw-B	Bee	68.7	30.8	17.4	82.1	<i>68.7</i>
	Pretzel	84.3	64.8	40.2	83.6	<i>83.2</i>
VB/16, VB/32, VL/16	Bee	69.0	28.8	34.8	19.2	<i>47.6</i>
	Pretzel	83.4	51.1	68.4	25.4	<i>73.8</i>

Table 3.9 Transferability results of a generator trained on ViT ensembles using 80×80 patches. Format follows Table 3.8.

Ensemble	Target	VB/16	VB/32	VL/16	Sw-B	Ens. ASR
VB/32, VL/16, Sw-B	Bee	21.0	46.6	44.4	90.0	<i>72.5</i>
	Pretzel	85.5	77.9	88.0	96.0	<i>92.5</i>
VB/16, VL/16, Sw-B	Bee	85.1	1.8	65.1	94.3	<i>89.7</i>
	Pretzel	93.3	39.6	89.5	96.4	<i>94.6</i>
VB/16, VB/32, Sw-B	Bee	70.7	45.8	14.1	92.6	<i>81.5</i>
	Pretzel	93.1	79.3	50.5	96.8	<i>94.9</i>
VB/16, VB/32, VL/16	Bee	82.4	56.0	53.6	40.6	<i>70.5</i>
	Pretzel	93.4	77.8	88.0	62.8	<i>91.4</i>

²We note that in the training process we use soft voting, rather than hard voting. This helps achieve higher ASR values for the ensembles.

To visualize the tables better, we include Figure 3.15 below, which summarizes the effectiveness of ensemble training for transferability. We plot attack success rates of generators trained on all four ensembles and tested against the unseen model. It includes results for both target classes and patch sizes.

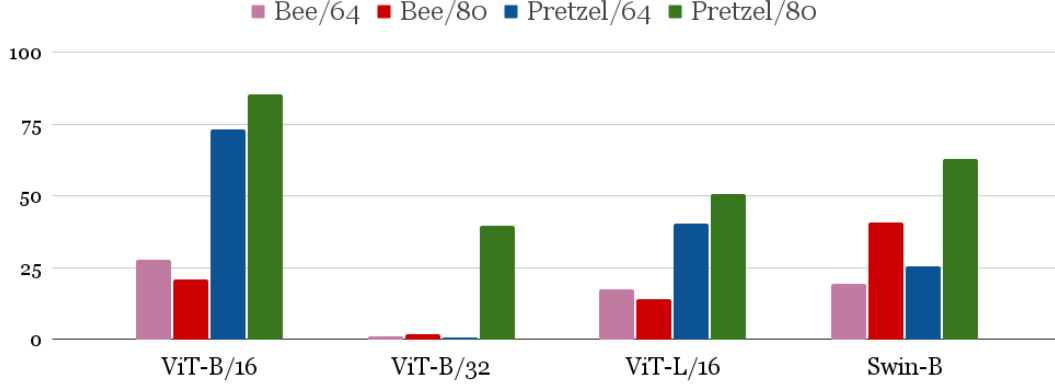


Figure 3.15 Attack success rates of generators trained on different ViT ensembles and evaluated on the held-out model. The plot shows results for both target classes (*Bee* and *Pretzel*) and patch sizes (64×64 and 80×80).

This figure clarifies that, in most cases, ensemble training improves transferability to unseen models. This is most noticeable for target class *Pretzel* - confirming that generators trained on this class tend to generalize more easily and achieve higher transferability rates. For example, the ensemble tested against ViT-B/16 achieves ASR of **73-85%**, which is quite close to the success rates yielded when training *directly* on that model.

On the other hand, for the target class *Bee*, we see that generators across all ensembles struggle to generalize, consistently staying below **30%** ASR. This difference between target classes emphasizes how important the choice of target is when aiming for generalizable adversarial patches.

To illustrate how the generated patches differ, we include Figure 3.16 below. This figure compares examples of patches trained on single ViTs (top row) with patches trained on ensembles excluding the corresponding model (bottom row).

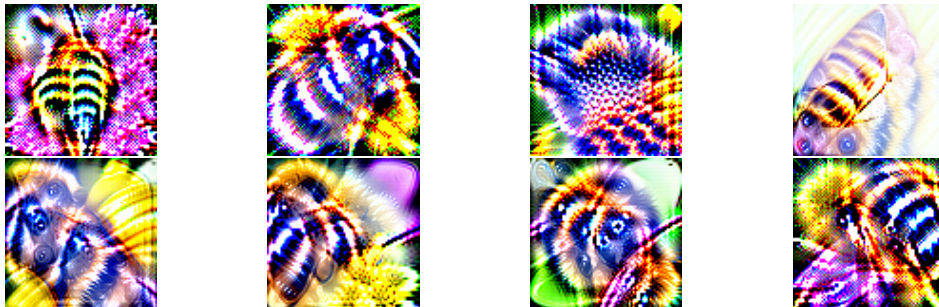


Figure 3.16 Comparison of 80×80 adversarial patches generated for target class *Bee*. The top row shows patches trained on individual models (ViT-B/16, ViT-B/32, ViT-L/16, Swin-B). The bottom row shows patches trained on ensembles that exclude the corresponding model from the top row (for example, the first patch in the second row was trained on an ensemble of ViT-B/32, ViT-L/16, Swin-B models).

This visual comparison helps illustrate how ensemble training can influence the structure of the generated patches. It is especially noticeable in the ensembles involving Swin-B: in the top row, the patch generated during training on Swin-B is quite different from the rest of the patches. If observed closely, we can see the "eyes" of a *Bee* – a visual feature that does not appear in the patches of other ViTs. In the second row, we see this feature more subtly in the patches trained on ensembles that include Swin-B. The exception is the very last patch, which was trained on the ensemble of ViT-B/16, ViT-B/32, ViT-L/16, and does not show this characteristic at all.

This observation brings us to our next point: adversarial patches against Swin-B actually perform worse in the ensemble-trained setting compared to single-model training. This can be explained by the fact that the ensemble composed entirely of ViTs (ViT-B/16, ViT-B/32 and ViT-L/16) encourages the generator to learn the inherent architectural traits of these type of models – traits which are very different from those of Swin-B.

Another model that appears very difficult to attack as a test model is ViT-B/32. Success rates against this model often drop significantly (e.g., **0.8%** in Table 3.8 and **1.8%** in Table 3.9 for the *Bee* target), particularly when it is excluded from the ensemble.

In summary, ensemble training enhances cross-model transferability, especially when the ensemble is composed of structurally similar models. Including outlier architectures like Swin-B improves performance on that specific model but may hinder generalization across the ViT family.

3.4.2 Inter-Family Transferability

We now extend our evaluation to **inter-family** transferability, where we test the generalizability of adversarial patches across different model architectures. In this framework we, again, differentiate between single-model transferability and ensemble trainings. In each setting, we evaluate how adversarial attacks trained on ViT models perform against CNN models.

Additionally, we explore a more complex setup: training on a *mixed* ensemble of both ViT and CNN models, which we hope leads to more robust and transferable attacks.

In this section, our experiments include a diverse set of models:

- ViT-B/16, ViT-B/32, ViT-L/16, Swin-B from Vision Transformers;
- ResNet50, ResNet101, ResNet152, and VGG16-BN among CNNs.

Single Model Transferability

As before, we begin with the single-model training setting. In Tables 3.10 and 3.11, we present results for training on single ViT models and testing on a range of CNNs, for 64×64 and 80×80 patches respectively.

The table format remains the same as in our previous single-model transferability experiments — each cell contains both the average ASR (across five generator runs) and the best observed ASR, for two representative target classes: *Bee* (difficult) and *Pretzel* (easier).

Table 3.10 ASR results for 64×64 adversarial patches transferred from ViT models to CNNs. Each cell shows average/best ASR across five runs. The ASRs of best-performing generators are in **bold** for more clarity.

Train Model	Target	ResNet50	ResNet152	VGG16-BN
ViT-B/16	Bee	3.4/ 3.9	4.5/ 5.5	0.1/ 0.1
	Pretzel	17.7/ 18.7	23.7/ 27.4	0.2/ 0.3
ViT-B/32	Bee	0.4/ 0.5	0.6/ 1	0.1/ 0.1
	Pretzel	9.8/ 11.2	19/ 24.8	0.5/ 1
ViT-L/16	Bee	0.4/ 1.1	1.5/ 3.5	0.1/ 0.1
	Pretzel	7.9/ 11.4	12.6/ 21.3	0.1/ 0.1
Swin-B	Bee	0.9/ 2.3	1.3/ 2.1	0.1/ 0.1
	Pretzel	8.4/ 14.9	10.6/ 21.2	0.7/ 1.5

Table 3.11 ASR results for 80×80 adversarial patches transferred from ViT models to CNNs. Format follows Table 3.10.

Train Model	Target	ResNet50	ResNet152	VGG16-BN
ViT-B/16	Bee	5.4/ 7.4	7/ 10	0.1/ 0.1
	Pretzel	24.4/ 32	28/ 36.8	0.1/ 0.2
ViT-B/32	Bee	2/ 3.2	3.5/ 4	0.1/ 0.1
	Pretzel	15.3/ 18.1	20.5/ 23.4	0.2/ 0.4
ViT-L/16	Bee	0.4/ 1	1.8/ 3.1	0.1/ 0.1
	Pretzel	16.1/ 29.7	21.1/ 50.6	0.1/ 0.2
Swin-B	Bee	2.2/ 4.2	2.4/ 5.1	0.1/ 0.1
	Pretzel	2/ 3.6	1.9/ 3.9	0.8/ 1.7

These tables show a significant drop in transferability compared to the intra-family experiments. This is the most noticeable with VGG16-BN model. Even though with direct attacks it was one of the most vulnerable models, in this case, VGG16-BN shows very high robustness against transferred attacks (below **2%** ASRs for both target classes and patch sizes). This highlights the big architectural gap between ViTs and CNNs, especially VGG models.

ResNets demonstrate moderate vulnerability to transferred attacks - especially for the easier class *Pretzel*. Interestingly, ResNet152, which was more robust than ResNet50 in direct attacks, appears more susceptible to transferred attacks. This might suggest that the deeper ResNet architecture may share more feature extraction similarities with transformer models than the shallower ones.

Among the ViT models, ViT-B/16 achieves the strongest transferability with 64×64 patches, and ViT-L/16 with 80×80 . On the other hand, generators trained on Swin-B show very low transferability, and, even though counterintuitive, it decreases even more with 80×80 patches. This may be caused by Swin-B’s unique shifted window attention mechanism, which processes features differently than standard ViTs. The larger patches might overfit to the vulnerabilities that are specific to the source model, and not generalize to convolutional architectures.

These results once again highlight the limitations of single-model transferability. In the next section, we explore ensemble-based approaches to investigate if it improves attack effectiveness.

Ensembling

Now we evaluate how well adversarial patches generalize when trained on an ensemble of models from one architecture and tested on another. We aim to explore if the exposure to multiple ViTs may help the generator capture more universal features that could transfer to CNNs.

Results are reported in Tables 3.12 and 3.13 for patches of size 64×64 and 80×80 , respectively.

Table 3.12 Transferability results of generators trained on ensembles of ViT models and tested on CNNs, using 64×64 adversarial patches. Results are reported for two representative target classes *Bee* and *Pretzel*. The last column lists ASR results of the ensembles themselves.

Ensemble	Target	ResNet50	ResNet152	VGG16-BN	Ens. ASR
VB/16, VB/32	Bee	3.3	5.2	0.1	<i>55.8</i>
	Pretzel	24.5	45.4	3.7	<i>79.1</i>
VB/16, VL/16	Bee	0.9	2.0	0.1	<i>57.7</i>
	Pretzel	18.3	26.7	0.4	<i>83.7</i>
VB/32, VL/16, Sw-B	Bee	7.1	8.8	0.2	<i>43.9</i>
	Pretzel	22.8	39.8	12.9	<i>68.9</i>
VB/16, VB/32, VL/16	Bee	3.3	5.4	0.1	<i>47.6</i>
	Pretzel	39.0	54.4	0.4	<i>73.8</i>

Table 3.13 Transferability results of generators trained on ensembles of ViT models and tested on CNNs, using 80×80 adversarial patches. Table follows the format of Tab. 3.12 above.

Ensemble	Target	ResNet50	ResNet152	VGG16-BN	Ens. ASR
VB/16, VB/32	Bee	15.3	17.4	0.1	<i>80.5</i>
	Pretzel	36.8	52.4	3.1	<i>90.8</i>
VB/16, VL/16	Bee	6.0	12.5	0.1	<i>83.5</i>
	Pretzel	20.6	31.9	0.2	<i>92.6</i>
VB/32, VL/16, Sw-B	Bee	10.5	12.0	0.1	<i>72.5</i>
	Pretzel	73.9	86.7	37.1	<i>92.5</i>
VB/16, VB/32, VL/16	Bee	8.0	13.2	0.1	<i>70.5</i>
	Pretzel	61.3	75.7	1.3	<i>91.4</i>

These tables reveal that ensembling, just like in the intra-family setting, consistently boosts transferability, particularly for the easier target class Pretzel. For example, when patches are trained on the ensemble of ViT-B/16, ViT-B/32, and ViT-L/16, the ASR on ResNet152 reaches **54.4%** for 64×64 patches and increases even more to **75.7%** for 80×80 patches; compared to the single-model setting, where each of the models from this ensemble achieved on average around **30%** transferability rates.

The most effective ensemble appears to be the one consisting of ViT-B/32, ViT-L/16 and Swin-B. Even though the ASR of the ensemble itself is not the highest compared to other combinations of ViT models, it has the highest transferability rates on CNNs. For example, in 80×80 patch and *Pretzel* target class setting, this ensemble achieves ASRs of **73.9%** and **86.7%**, for ResNet50 and ResNet152

respectively. We see a significant improvement with VGG16-BN as well – this ensemble achieves **12.9%** and **37.1%** success rates (in 64×64 and 80×80 patches, respectively), which is a huge boost from the results shown in previous experiments. This suggests that exposing the generator to a wider range of ViT architectures enables it to capture more transferable features.

In summary, the ensemble-based patches outperform the single-model ones across most settings. These findings once again demonstrate that architectural diversity in the training ensemble is crucial for enhancing transferability not only within the model architecture, but also across different model families.

Mixed-Family Ensembles

In this final section, we explore mixed-family ensembling, where the generator is trained on a combination of ViT and CNN models. Since Vision Transformers and CNNs process images in different ways – self-attention over tokens vs. hierarchical feature extraction – we believe that training generators on the ensembles containing both model types will provide interesting insight into adversarial transferability.

In this section we evaluate six ensembles in total: five that contain a diverse mix of ViT and CNN models, and one additional ensemble composed entirely of CNNs:

- ViT-B/16 and ResNet50;
- ViT-B/16, ViT-B/32 and ResNet101;
- ViT-B/32, ResNet101 and ResNet152;
- ViT-B/16, ViT-B/32, Swin-B and ResNet152;
- ViT-B/16, ResNet50, ResNet101 and ResNet152;
- ResNet50, ResNet101 and ResNet152.

Before presenting the ASR tables, we provide visual representations of adversarial patches for each ensemble. Fig. 3.17 shows patches generated during training for the target class *Bee*:



Figure 3.17 Adversarial patches of size 80×80 and target class *Bee*, trained on mixed-ensembles. The order of patches follows the list of ensembles above, from left to right, and top to bottom.

While Fig. 3.18 demonstrates the adversarial examples generated during training for the target class *Pretzel*.



Figure 3.18 Adversarial patches of size 80×80 and target class *Pretzel*, trained on mixed-ensembles. The order of patches follows the format of Fig. 3.17 above.

These figures show that adversarial patches targeting the *Bee* class tend to exhibit more abstract and "high-frequency" features, whereas patches targeting *Pretzel* look more clearly structured and shaped (with easily identifiable pretzel shapes). It is interesting that for both target classes, patches corresponding to the ensemble of ViT-B/16, ResNet50, ResNet101 and ResNet152 models (second patch in the bottom row in each figure) stand out with a more focused and distinct design. As opposed to the ensemble containing only CNNs, which seem more diffused and textured. This visual differences motivate us to examine the transferability results across all ensembles and target models.

Below we present Tables 3.14 and 3.15 that summarize the results of experiments conducted on selected ensembles. We follow the previous experimental setup and report ASRs for two representative classes, *Bee* and *Pretzel*, as well as for both patch sizes.

Table 3.14 Transferability results of 64×64 patches trained on a joint ViT+CNN ensemble. Target class names are shortened for compactness: "B" stands for "Bee" and "P" for "Pretzel". Additionally, for clarity, ASR scores under models that are included in the ensemble are in *italics*.

Ensemble	TC	VB/16	VB/32	VL/16	Sw-B	RN50	RN152	VGG
VB/16, RN50	B	<i>74.9</i>	0.7	8.4	28.8	<i>31.8</i>	6.72	1.4
	P	<i>86.2</i>	13.0	38.9	62.2	<i>79.3</i>	58.3	0.9
VB/16, VB/32, RN101	B	<i>56.6</i>	<i>21.3</i>	6.2	23.9	1.9	4.7	0.2
	P	<i>84.7</i>	<i>63.4</i>	43.2	51.1	64.8	87.3	23.1
VB/32, RN101, RN152	B	26.4	<i>32.3</i>	8.33	30.5	51.4	<i>72.1</i>	3.5
	P	49.4	<i>64.0</i>	37.8	62.0	69.7	<i>94.4</i>	34.9
VB/16, VB/32, Sw-B, RN152	B	<i>53.2</i>	<i>15.8</i>	6.4	<i>68.2</i>	17.9	<i>40.9</i>	0.2
	P	<i>85.2</i>	<i>62.0</i>	42.0	<i>89.2</i>	67.4	<i>90.7</i>	35.1
VB/16, RN50, RN101, RN152	B	<i>13.7</i>	0.5	0.8	17.8	<i>18.4</i>	<i>22.1</i>	0.1
	P	<i>81.7</i>	27.4	40.9	67.7	<i>88.2</i>	<i>92.7</i>	20.1
RN50, RN101, RN152	B	0.9	0.2	0.7	31.1	<i>79.3</i>	<i>58.8</i>	4.7
	P	35.3	12.5	25.1	69.9	<i>93.3</i>	<i>94.5</i>	36.9

Table 3.15 Transferability results of 80×80 patches trained on a joint ViT+CNN ensembles. Table follows the format of Tab. 3.14 above.

Ensemble	TC	VB/16	VB/32	VL/16	Sw-B	RN50	RN152	VGG
VB/16, RN50	B	<i>86.0</i>	0.9	21.1	43.0	<i>88.7</i>	42.8	2.1
	P	<i>91.7</i>	16.3	51.3	65.5	<i>91.0</i>	67.0	12.4
VB/16, VB/32, RN101	B	<i>28.0</i>	<i>36.0</i>	2.9	26.6	4.7	7.9	0.1
	P	<i>92.44</i>	<i>79.0</i>	51.0	57.0	85.5	93.1	16.0
VB/32, RN101, RN152	B	1.2	<i>38.3</i>	1.6	17.2	0.8	<i>47.3</i>	0.1
	P	73.4	<i>79.5</i>	38.8	72.5	88.0	<i>97.7</i>	32.6
VB/16, VB/32, Sw-B, RN152	B	<i>20.5</i>	<i>34.5</i>	3.6	<i>80.5</i>	5.4	<i>54.1</i>	0.1
	P	<i>92.4</i>	<i>77.8</i>	63.4	<i>96.0</i>	80.0	<i>95.5</i>	20.2
VB/16, RN50, RN101, RN152	B	<i>73.3</i>	0.4	4.6	38.2	<i>57.7</i>	<i>57.1</i>	2.3
	P	<i>92.1</i>	42.1	63.8	74.9	<i>95.5</i>	<i>96.8</i>	27.4
RN50, RN101, RN152	B	0.1	0.2	0.3	28.6	<i>52.1</i>	<i>57.4</i>	0.1
	P	47.9	2.9	24.1	69.6	<i>96.8</i>	<i>96.4</i>	48.3

While the tables contain detailed information for all possible setups, to better understand the trends, we focus our analysis on the 80×80 patch results and *Pretzel* target class. This choice is due to two key observations: first, 80×80 patches consistently outperform 64×64 patches across all ensembles in the mixed-ensemble setting; and second, *Pretzel* is a relatively easier class to generalize to unseen models, which allows us to discuss more consistent and clear trends.

Therefore, for both clarity and comparability with previous experiments, we narrow our analysis down to this setup. To further aid our analysis, we also present two charts (3.19 and 3.20) that visualize the transferability rates of adversarial patches across ViTs and CNNs, respectively.

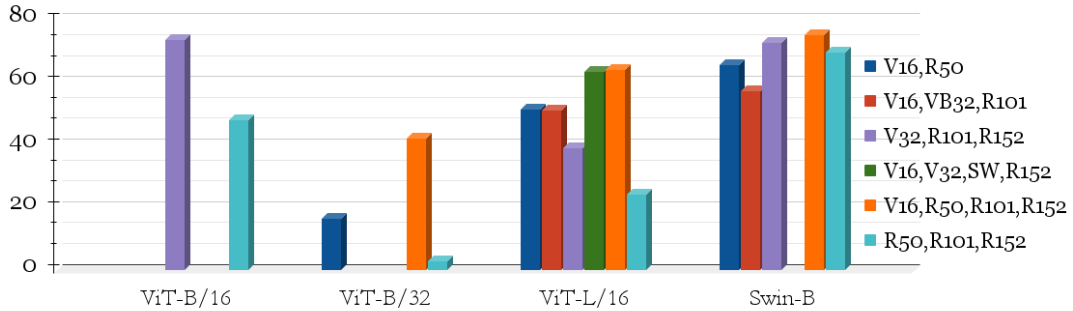


Figure 3.19 ASR values against ViT target models for adversarial patches generated from mixed-family ensembles using 80×80 patches and target class *Pretzel*.

Figure 3.19 demonstrates some interesting patterns in transferability performance of mixed ensembles against ViTs. Among all models, **ViT-B/32** appears to be the most robust against mixed-family attacks, with the only notable ASR of **42.1%** coming from the ensemble containing ViT-B/16, ResNet50, ResNet101 and ResNet152. Interestingly, this same ensemble achieves a much higher success rate of **63.8%** on the **ViT-L/16** model, outperforming the all-ViT ensemble (ViT-B/16, ViT-B/32, Swin-B) which achieved **50.5%** ASR (see Table 3.9).

Another mixed ensemble that outperformed the all-ViT generator is ViT-B/16, ViT-B/32, Swin-B and ResNet152, which reached **63.4%** ASR against **ViT-L/16**.

This highlights that including a CNN architecture in the ensemble improves generalization, especially against deep ViT models like ViT-L/16.

As expected from earlier experiments, **Swin-B** remains the most vulnerable model to transferred attacks. Previously, 80×80 patches consistently achieved below **50%** in single-model transferability experiments (except ViT-B/16, with 68%; see Table 3.7), and **62.8%** for the ViT ensemble ViT-B/16, ViT-B/32, ViT-L/16 (see Table 3.9). Now, we see that mixed ensembles lead to some improvements. For example:

- ViT-B/16 and ResNet50 achieve **62.2%** ASR;
- Adding two more CNNs: ResNet101 and ResNet152 increases the ASR even more to **67.7%**.
- Most interestingly, the highest ASR of **69.9%** against Swin-B is achieved by a generator trained on an ensemble that does not contain ViT models at all – i.e. ensemble consisting of all ResNets (ResNet50, ResNet101 and ResNet152). This emphasizes the surprising strength of CNN-only ensembles in generating transferable patches.

CNN-only ensemble performs relatively well on **ViT-B/16** model too, with the ASR slightly over **45%**.

Now, let us examine the transferability results of mixed-family ensembles against CNN models. Figure 3.20 highlights the most important results from the detailed tables above.

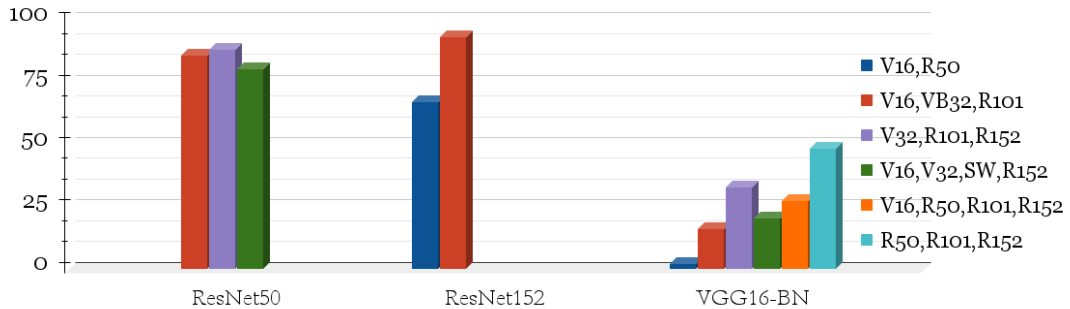


Figure 3.20 ASR values against CNN target models for adversarial patches generated from mixed-family ensembles using 80×80 patches and target class *Pretzel*. Note, that ASR values for ensembles that included the target model are not present in the chart.

As expected, **VGG16-BN** remains the most robust CNN model, although we still see slight improvements in ASRs when compared to those reported in all-ViT ensemble transferability experiments (see Tables 3.12 and 3.13). The most effective attack against this model is achieved by a generator trained on all-CNN ensemble (ResNet50, ResNet101 and ResNet152), with ASR **48.3%**.

For other ResNets we see more promising results, with each transferred attack achieving higher than **60%** success rate. In particular, all three valid ensembles (i.e. ensembles that do not contain ResNet50 itself), reached an ASR around **80%**. The most successful among them was generated by a generator trained on the ensemble of ViT-B/32, ResNet101 and ResNet152. This result is intuitive, since it includes two ResNet models, compared to the other two ensembles which include only one.

3.4.3 Summary

The findings discussed in previous sections revealed important information about the robustness and generalizability of G-patch attacks. Our results show that in most cases, intra-family transferability is more effective than inter-family transferability. This emphasizes that the model architecture greatly influences how the generator is trained and how it produces adversarial patches. ViTs and CNNs are fundamentally different in processing inputs, and they have distinct vulnerabilities that the generator can exploit for successful attacks.

Within our intra-family transferability experiments, we observed that patches trained on ensembles of ViTs transfer more effectively to other ViTs than patches trained on single models.

It is worth noting that while intra-family transferability is inherently stronger than inter-family transferability, particularly with generators trained on single-model and ensemble (non-mixed) settings, **mixed-ensemble** training stands out as a powerful approach for generating more universal adversarial patches. The architectural diversity in such ensembles clearly enables generators to capture more general vulnerability patterns across model families. This suggests that not limiting the ensembles to only one model architecture can help attacks easily generalize not only within that architecture, but also across different families.

Our work also reveals that the choice of target class and patch size are important factors in transferability success. The experiments show a consistent trend where larger patches demonstrate superior transferability. Additionally, generators trained for the target class *Pretzel* consistently outperformed those trained on *Bee*. Based on these findings, we can recommend that when attempting to transfer an attack, using 80×80 patches with relatively easier target classes like *Pretzel*, and training on a diverse mix of models from different architectures yields optimal results.

4 Mini Patch Attacks

Inspired by our implementation of the *Random Position Patch Attack*, we propose a new approach called *Mini Patch Attacks*. In this approach, we maintain the overall structure of the original network, but with a key difference: as the name of the attack suggests, the size of the generated patch is significantly smaller.

While in the previous implementation we generated patches of size 64×64 or 80×80 , in the Mini Patch Attacks we aim to use much smaller patches – specifically, we experimented with patch sizes of 16×16 and 32×32 . Of course, we limit the number of mini patches so that the overall perturbed part remains about 8% of the original image. For example, we deploy patches of size 16×16 at 8 different points of the image, whereas for 32×32 patches we limit that number to 4. This ensures that the total modified area remains around 8%, matching the coverage of a single 64×64 G-patch.

The logic for training the generator remains the same as in the original approach. By adjusting the kernel size, stride, and padding parameters of the convolutional layers in the Generator, we can now create mini patches. The generator is still trained using adversarial loss, with the objective of misleading victim models and misclassifying modified images as target classes. Just like with G-patches, we again train the *mini-patch* generators over 40 epochs, and save them temporarily. We then evaluate the saved generators on the dataset using the same 5 fixed random noise vectors and select the best-performing one (with the highest average ASR value) for testing later on.

We train on the same 5 target classes: *Maltese Dog*, *Bee*, *Cassette*, *Hockey Puck* and *Pretzel*, and repeat the training process 3 times for better generalizability of the attacks.

Regarding the Deployer, we slightly modified the original mechanism so that it deploys the adversarial patch(es) at a given number of positions across the image. These modifications raise several important questions: Does the deployer still maintain randomness? Should we reuse the same mini patch across all positions, or generate a unique one for each?

In the following sections, we explore and discuss the different approaches we tested within the context of Mini Patch Attacks.

4.1 Random Position Mini Patches

The first approach we explore is (sort of) an extension of the original G-patch attack. As mentioned above, the training process remains largely untouched – we only modify the deployment mechanism to allow patch placement at several positions. It is important to note that in this approach, the randomness of the patch position is preserved. Therefore, the binary mask M is generated such that it contains 1's at n randomly chosen patch positions and 0's elsewhere. The deploy function remains the same: $T(M_p, x) = M_p + (1 - M_p) \cdot x$.

In this chapter we will be using 16×16 patches against all victim models, except ViT/B-32. This model itself tokenizes the image into 32×32 segments, therefore, we also use adversarial examples of the same size to align with the internal token resolution.

Below, we present the results of randomly placed *mini patches* (Table 4.1). We evaluate each setting over 3 runs per target class. Again, we run the experiments not only across ViTs models, but also several CNN-based architectures.

Table 4.1 ASR scores, evaluated over 3 runs per target class. Target models are abbreviated for compactness (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50).

TC	It.	VB/16	VB/32	VL/16	Sw-B	RN50	RN152	VGG
Maltese Dog	1	7.11%	23.88%	0.60%	44.63%	0.15%	0.14%	74.99%
	2	6.93%	8.85%	0.45%	0.11%	0.11%	0.14%	73.76%
	3	6.92%	0.16%	0.52%	61.56%	0.13%	0.13%	73.25%
Bee	1	19.19%	17.13%	1.43%	72.93%	56.89%	30.28%	63.24%
	2	23.87%	16.96%	1.22%	70.24%	53.86%	43.33%	60.71%
	3	18.93%	17.24%	1.69%	60.53%	54.97%	28.36%	60.86%
Cassette	1	0.13%	1.34%	0.14%	0.56%	18.08%	11.14%	39.47%
	2	0.12%	1.46%	0.14%	42.95%	8.40%	12.37%	45.59%
	3	0.13%	1.41%	0.15%	42.67%	23.27%	0.13%	43.72%
Hockey Puck	1	37.78%	14.51%	3.34%	63.97%	27.43%	14.08%	55.64%
	2	36.55%	10.79%	0.35%	50.88%	28.51%	4.93%	55.95%
	3	39.89%	13.67%	3.86%	84.66%	25.86%	13.86%	52.71%
Pretzel	1	0.19%	64.15%	21.63%	0.10%	63.12%	0.48%	12.89%
	2	0.16%	63.90%	22.88%	0.11%	57.60%	42.38%	13.28%
	3	0.18%	63.34%	21.81%	0.11%	61.12%	38.47%	14.93%

This table shows several notable patterns. The most noticeable is the high variability across different runs for the same experimental configuration (same victim model and target class). For example, generators trained against Swin-B or ViT-B/32 on target class *Maltese Dog* are quite inconsistent, especially when compared to the results of the G-patch attacks. We hypothesize that this inconsistency is caused by the small size and higher sensitivity of mini patches. Since these patches are very limited in size (hence also limited in adversarial encodings), poor initialization is more likely to train a generator that is unable to learn effective perturbations. In other words, these attacks are more prone to getting "unlucky" in the initial stages and failing to converge to a strong generator.

To illustrate this better, we present the training logs for the unsuccessful generator targeting ViT-B/32.

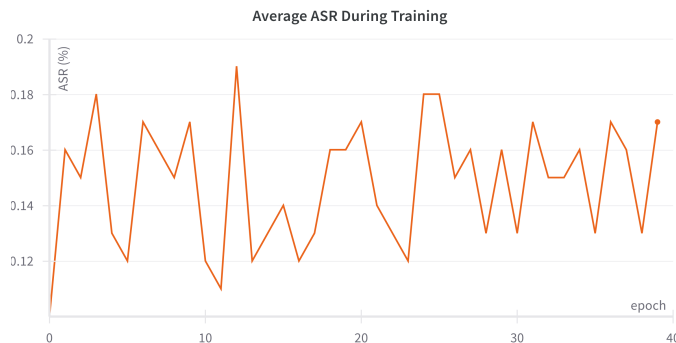


Figure 4.1 Average ASR during training of a generator against ViT-B/32 (target class: Maltese Dog).

Figure 4.1 above demonstrates the evolution of average ASR over training epochs for the target class *Maltese Dog*, that ultimately achieved only 0.16% ASR. This noisy curve suggests that the generator struggles to stabilize and find optimal perturbations.

We also visualize the difference between the unsuccessful (ASR 0.16%) and successful (ASR 23.88%) generators in the Figure 4.2 below. Although their initial patches (after epoch 1) look similar, their final outputs (after best-performing epochs) differ significantly. The unsuccessful generator barely changes the patch over time, while the successful one converges to a visibly distinct and more effective patch.



Figure 4.2 Comparison between unsuccessful (left: epoch 1 and 16) and successful (right: epoch 1 and 38) patches for ViT-B/32 targeting "Maltese Dog". Both were trained under the same settings.

These visualizations once again confirm our hypothesis that smaller patches are simply more prone to getting "unlucky" and failing to stabilize and converge to an optimal generator.

To illustrate what the mini patches look like in practice, we now present example adversarial images that managed to deceive victim models Swin-B and ViT-B/32 into classifying them as the target class *Bee*.

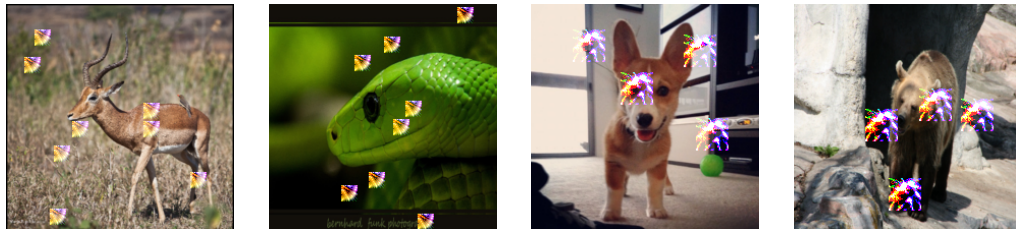


Figure 4.3 Example adversarial images that were misclassified as the target class "Bee". Original labels were: *Gazelle*, *Green Mamba*, *Welsh Corgi*, *Bear* (left to right). The first two images (Gazelle and Green Mamba) display 16×16 adversarial patches trained on Swin-B; whereas the last two (Welsh Corgi and Bear) demonstrate 32×32 patches trained on ViT-B/32.

It is interesting that the inconsistency across different runs of the same experimental setup is more noticeable in certain target classes, like *Maltese Dog* and *Pretzel*. This could be because small patches might not be able encode sufficient information to mislead the model. On the other hand, simpler, or more spatially distinct classes like *Cassette* or *Bee* achieve more stable results. Figure 4.3 further confirms this, as we see how 16×16 patches compared to 32×32 ones resemble the target class less, as they do not have as much spatial capacity to do so.

Despite this variability, we can still detect some clear trends among victim models. Swin-B and VGG16-BN remain the most susceptible models against adversarial examples. Even with randomly placed 16×16 patches, ASR values

often exceed 60-70% on these models. This was expected, as it aligns with their performance against G-patches (discussed in Chapter 3), where they were also the most vulnerable.

ViT-B/32 seems generally more robust, except for the target class *Pretzel*, where the ASR value is around 64%. This suggests that the generator successfully encoded sufficient information into the 32×32 patch. We note that this result is comparable to those achieved by larger 64×64 G-patches.

ViT-L/16 remains the most robust model overall, with ASR values consistently below 4% for the majority of target classes.

Among the CNNs, ResNet152 is more robust than ResNet50 for most classes, although both show moderate vulnerability to *Bee* and *Pretzel* attacks.

Finally, we present a bar plot below in Figure 4.4 to sum up the performance of random mini patches, and compare it to the results achieved by G-patches.

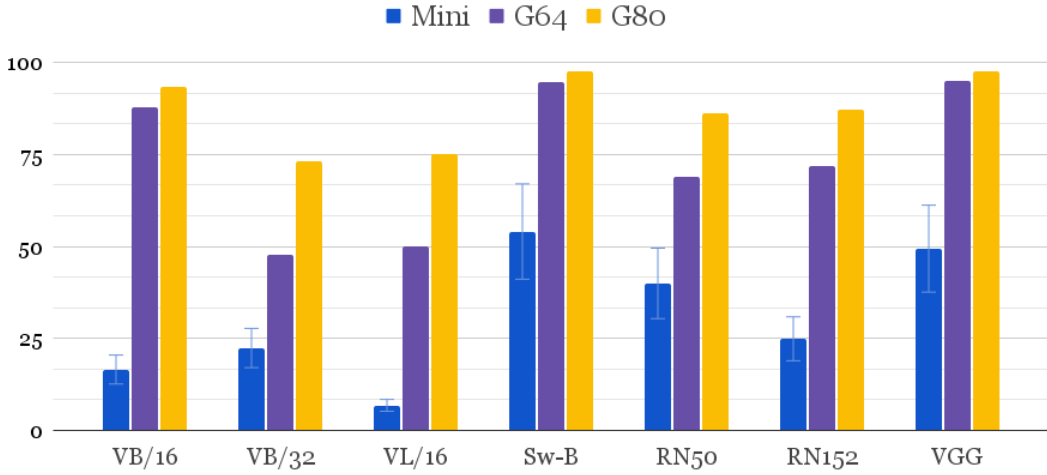


Figure 4.4 Comparison of average ASRs across victim models for Mini Patch (blue), G-Patch 64×64 (purple), and G-Patch 80×80 (gold) attacks. ASR values are averaged over five target classes. For Mini Patch, error bars indicate the standard deviation of the average ASR across those five target classes, highlighting variability in per-class performance.

This demonstrates that mini patches, though limited in spatial coverage, can still effectively attack both ViTs and CNNs. However, their performance is less reliable and more variable than larger G-patches. Their success depends strongly on the patch size, the model architecture, and the complexity of the target class.

The inconsistency observed in certain experiments further motivates us to explore more structured training and deployment strategies. In particular, we introduce a more structured placement strategy, targeting semantically or structurally important regions of the image. Instead of deploying patches at purely random positions, we can exploit the way vision models process images and strategically place adversarial patches at key feature locations to maximize their impact.

Even though the main advantage of the original attack was the randomness of patch placement - especially for real-world deployment - structured and targeted placement is still an interesting approach to explore, and we introduce several strategies to do so.

4.2 Exploiting Critical Points

In this chapter, we mainly explore a different deployment mechanism by placing adversarial patches at *critical points* in the image. Critical points are exactly the above-mentioned key feature locations, although they differ depending on the training/target model’s architecture, and patch alignment with the tokenization process.

For example, ViT/B-16 uses tokens of size 16×16 pixels, while ViT/B-32 uses 32×32 . On the other hand, Swin transformer models use a hierarchical shifting window mechanism, where self-attention is computed within local windows that shift across layers. Therefore, defining critical points in Swin models is challenging, and we keep this in mind as we proceed with experiments.

Regarding patch alignment with image tokens, we would like to differentiate between two main approaches:

- a) Centering the adversarial examples at token ”corner” points, affecting 4 adjacent image patches simultaneously;
- b) Placing adversarial patches within the tokens (so that one patch affects only one token). In particular, we will effectively replace the transformer token with an adversarial patch.

In each case, the training process remains the same as in our previous experiments, except that patches are now positioned *precisely* at critical points. However, to still maintain some randomness and avoid overfitting the generator on the most sensitive areas of the image, we *randomly* select a subset of critical points in each iteration.

4.2.1 ”Corner” points

A particularly intuitive strategy is targeting corner points of image tokens. Since ViTs process images by dividing them into non-overlapping patches, we think that targeting the attack at intersection points between four adjacent tokens should have a more significant impact on model predictions than random placement.

Centering the adversarial example at the corner point essentially results in attacking 4 separate tokens simultaneously. This is illustrated in the Figure 4.5:

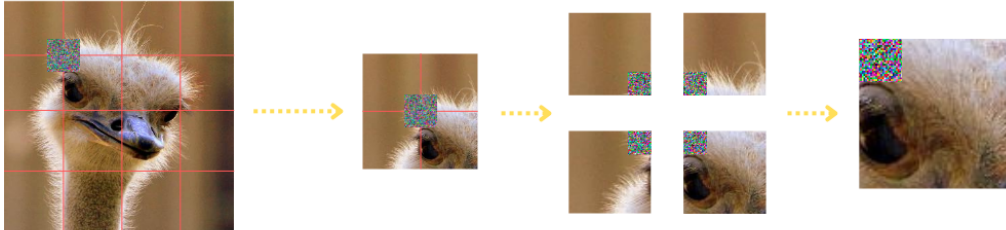


Figure 4.5 Adversarial patch of size 32×32 px. centered at a *corner* point of the image (red lines do not represent the exact tokenization of ViT models, for the sake of better visualization, tokens here are of size 56×56). This results in attacking 4 tokens with 8×8 size adversarial patches.

To validate this approach, we conducted experiments for the same five target classes (*Maltese Dog*, *Bee*, *Cassette*, *Hockey Puck*, and *Pretzel*). Even though this attack is inherently designed to target and exploit the unique architecture of ViT models with **fixed-size, non-overlapping** patches, we still evaluate it against both CNNs and Swin-B models. It is interesting to see if the more structured nature of patch placement achieves high ASRs across different architectures.

Additionally, we note that we use 16×16 patches against all the models except ViT-B/32, in which scenario we again train 32×32 adversarial examples.

Below, we present the table (4.2) that summarizes the results of mini patches centered at *corner* points. We evaluate each setting over 3 runs per target class.

Table 4.2 ASR scores, evaluated over 3 runs per target class. Target models are abbreviated for compactness (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50).

TC	It.	VB/16	VB/32	VL/16	Sw-B	RN50	RN152	VGG
Maltese Dog	1	44.48%	29.64%	5.32%	76.82%	40.47%	0.15%	81.85%
	2	41.77%	14.34%	3.73%	88.47%	61.67%	51.3%	82.94%
	3	43.56%	22.89%	11.23%	85.10%	<i>0.09%</i>	<i>0.17%</i>	83.25%
Bee	1	68.66%	44.46%	9.52%	91.14%	74.40%	46.26%	51.48%
	2	64.64%	34.68%	2.00%	90.12%	75.84%	63.86%	<i>4.80%</i>
	3	65.11%	33.87%	6.98%	90.72%	73.73%	48.07%	62.19%
Cassette	1	7.91%	21.51%	1.43%	86.62%	<i>0.51%</i>	1.13%	57.21%
	2	1.69%	17.24%	0.15%	82.60%	25.33%	0.14%	55.67%
	3	0.14%	10.95%	1.99%	76.20%	<i>0.13%</i>	0.13%	46.10%
Hockey Puck	1	77.18%	24.48%	5.44%	91.93%	40.18%	22.61%	62.89%
	2	76.15%	13.01%	7.84%	92.67%	55.44%	29.41%	57.32%
	3	77.07%	15.29%	2.46%	94.24%	45.11%	21.75%	62.33%
Pretzel	1	35.42%	62.55%	18.77%	93.62%	74.63%	<i>0.09%</i>	55.84%
	2	29.46%	60.47%	7.25%	93.05%	77.71%	50.34%	60.80%
	3	21.59%	61.05%	4.18%	80.29%	78.52%	42.13%	58.40%

The results are also visualized in Figure 4.6, which summarizes averaged ASR scores (across 3 runs) and helps identify trends more clearly.

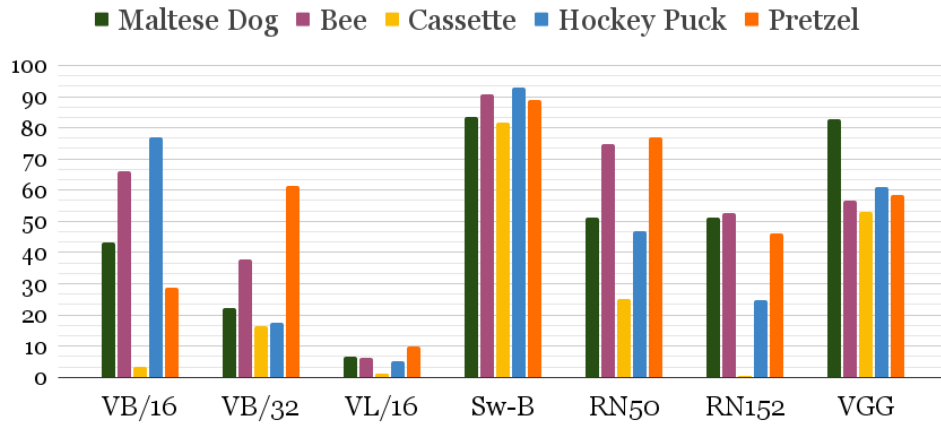


Figure 4.6 Average ASR of randomly placed mini patches; per target class and model.

These figures show a clear overall improvement in the consistency of generators, especially for ViT models. We rarely see "unlucky" generators in this setting,

and the few inconsistent cases appear only among CNNs. Generators trained on models like ViT-B/32 and Swin-B, which previously demonstrated high variability and often struggled to converge to a strong state, now are much more consistent across all target classes. One possible explanation for this is that, in this setup, the generator is trained with patches repeatedly placed at structurally important locations (i.e., corner points). This might allow the generator to learn more effectively, as it can exploit the predictable nature of patch positions. On the other hand, in the random deployment scenario, the generator had to generalize across a wide range of potential positions, which could lead to unstable convergence and lower success rates.

To verify this explanation, we present Figure 4.7, which shows a visual comparison of patches trained on the same victim model (ViT-B/32) and target class (*Bee*), in the random placement and *corner points* scenarios.

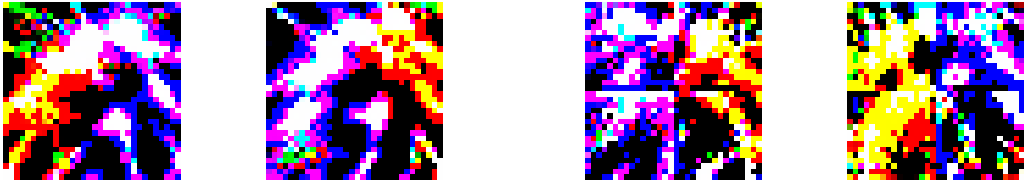


Figure 4.7 On the left: patches from two different runs in the random placement setting; on the right: patches from two different run in the *corner points* scenario. All generators were trained on the victim model ViT-B/32, for target class "Bee". All patches are of size 32×32 .

After examining the individual patches, the difference between the generators in both settings is quite clear. Patches from *corner point* attacks are visibly divided into four squares that would align with the 4 adjacent image tokens. Therefore, the generator is optimizing the patch to simultaneously influence adjacent tokens. This supports our hypothesis that the generator indirectly learns the adversarial patch’s positional role, which, in turn, helps to converge better and create more effective attacks.

We also include two examples of adversarial images to show what the patches look like in practice – aligned with the internal tokenization of Vision Transformers.



Figure 4.8 Example adversarial images that were misclassified as the target class "Bee". Original labels were: *Monkey*, *Spider* (left to right). Patches shown on these images were created by a generator trained on ViT-B/32.

In addition to improved consistency of the generators, these attacks also show a significant increase in ASRs, especially for ViT models. For example, ASRs

against ViT-B/16 increased by around **40%** for almost every target class. Similarly, patches centered at corner points showed an improvement of around **10-20%** in ASRs against ViT-B/32 and Swin-B. On the other hand, for the most robust ViT model - ViT-L/16 - we only see a slight increase for most target classes.

The improvements are also notable across CNNs. For example, ResNet50 shows increases from $\sim$ **50%** in random placement to up to $\sim$ **78%** in this structured setting. Although CNNs do not have token boundaries, the structured alignment appears to help generators during training to create more effective patches.

To better visualize these trends, we present Figure 4.9 below, which demonstrates average ASR values for each model, across all target classes.

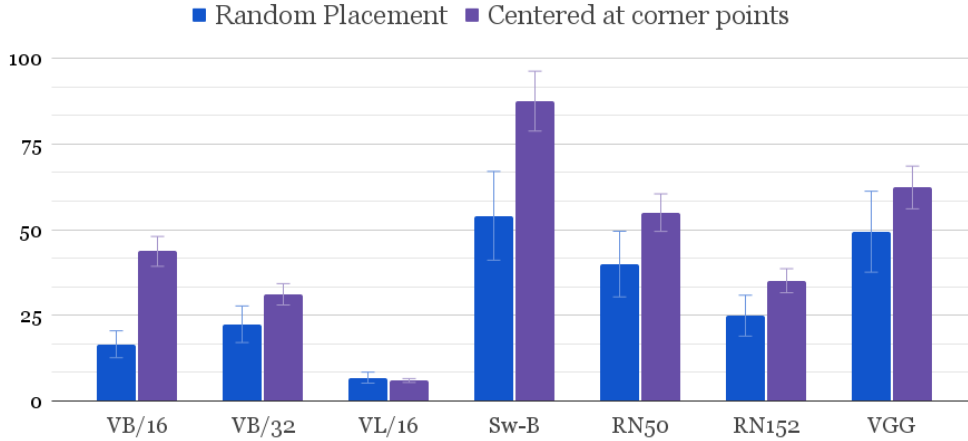


Figure 4.9 Comparison of average ASRs across victim models for *random placement* patches (blue) and *corner point* patches (purple). ASR values are averaged over five target classes. Error bars indicate the standard deviation of the average ASR across those five target classes, highlighting variability in per-class performance.

Additionally, we include a bar plot (Fig. 4.10) that compares both, random placement and corner points mini patches to original G-patches.

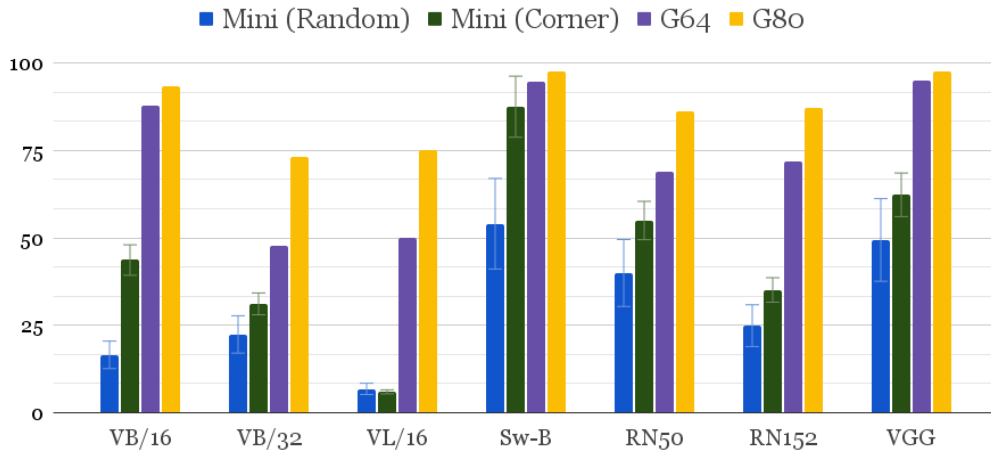


Figure 4.10 Comparison of average ASRs across victim models for *random placement* (blue) and *corner point* patches (green), to G-Patch 64×64 (purple), and 80×80 (gold) attacks.

This supports the hypothesis that attacking critical feature intersections is more effective than purely random positioning – especially for models that rely on token-based representations like Vision Transformers. In several cases, the results are almost as good as the ASRs achieved by 64×64 patches. For example, mini patches positioned at corner points manage to deceive Swin-B almost as consistently as G-patch of size 64×64 , as well as 80×80 . The latter is especially important since the overall coverage of the original image by adversarial examples is much larger. Results are comparable in the case of a larger and more robust model, ViT-B/32 as well. On the other hand, mini patches struggle to mislead ViT-L/16 in both placement settings. This suggests that deeper models like ViT-L/16 are robust against patches that are scattered around the image, as opposed to a single large patch deployed at a random position. The difference between ASRs in G-patch 64×64 and G-patch 80×80 further suggests that the size of the adversarial patch is also quite important – and the increase from 64 to 80 shows significant raise in the effectiveness of the attacks.

To sum it all up, corner point patches outperform random placement patches in almost every way, and even offer comparable results to the G-patch attacks. While random placement offers advantages in real-world deployment (e.g., unpredictability), structured placement provides more stable training and enables the generator to better align the perturbation with the model’s internal representations – especially in patch-based architectures like ViTs.

Inspired by this improvement, we further explore the positional vulnerabilities of Vision Transformers and propose another attack approach – placing adversarial examples within the borders of token patches.

4.2.2 Critical points within token patches

In this section we explore another promising approach of placing adversarial examples entirely within the token patches. This essentially means that a single adversarial patch will replace one image token. This ensures that each perturbation affects a single token rather than multiple adjacent ones, which in turn means that each token will be attacked by a larger adversarial patch, and this may yield different adversarial effects. Patch deployment principle is demonstrated in the Figure 4.11 below:

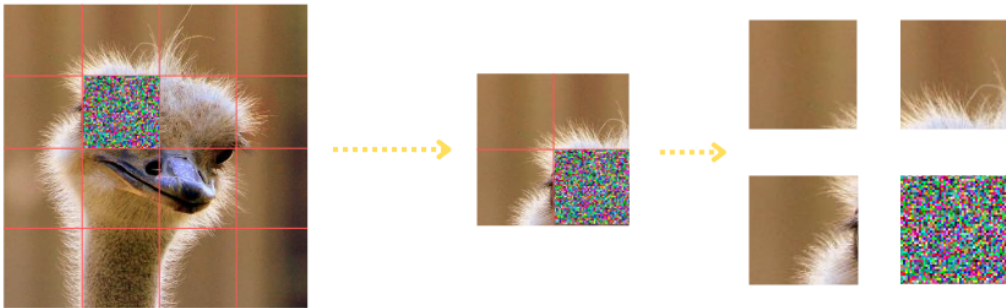


Figure 4.11 Adversarial patch placed entirely inside the token. Red lines do not represent the exact tokenization of ViT models, for the sake of better visualization, tokens here are of size 56×56 (and so is the example adversarial patch). This results in replacing 1 token entirely.

To validate this approach, we conducted experiments for only standard Vision Transformer models (i.e. models with fixed-sized, non-overlapping patches) – this is simply because CNNs are not token-based architectures and Swin-B employs a shifted window mechanism rather than fixed tokens. Thus, this approach would be essentially the same as centering the adversarial patches at *corner* points (see section 4.2.1).

Additionally, we note that we use 16×16 patches against all the models except ViT-B/32, in which scenario we again train 32×32 adversarial examples to replace the image token of the same size. To illustrate this, we present some of the adversarial patches created by generators trained for target class *Bee* in Figure 4.12 below:

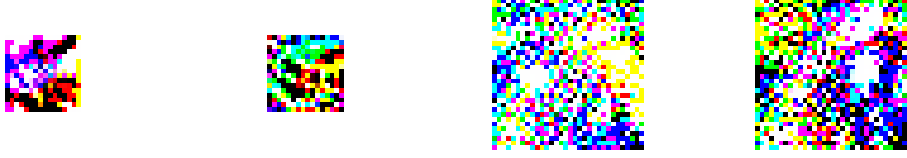


Figure 4.12 Adversarial patches trained on ViT-B/16, ViT-L/16 and ViT-B/32 (from left to right; the last two are patches from different runs on ViT-B/32 model).

We note that these patches look significantly different from the ones trained in previous setups - randomly placed patches and targeting the *corner points* of ViT tokens (see Fig. 4.7). This is especially noticeable in the patches trained on ViT-B/32 model. Unlike earlier variants, these do not visibly resemble the target class (*Bee*) and show a more pixelated, "diffused" appearance. It seems like the noise pattern appears more uniformly across the whole patch. This might be suggesting that replacing entire tokens encourages less localized, more broad perturbations.

To provide a more intuitive understanding and visualize what these patches look like in practice, we include Figure 4.13, which presents sample adversarial images where the token-replacement patches successfully fooled the victim models.



Figure 4.13 Example adversarial images that were misclassified as the target class "Bee". The first two images show patches that were created by a generator trained on ViT-B/16 model, while the last two – on ViT-B/32.

We see on these images that the adversarial patches are clearly aligned with the Vision Transformer grid (for both, ViT-B/16 and ViT-B/32 models). The patch placement is also visibly structured and consistent across the input images.

We now present Table 4.3, which summarizes the attack success rates for adversarial patches replacing individual ViT tokens. Again, we evaluate each configuration over 3 runs per target class.

Table 4.3 ASR scores for adversarial patches replacing ViT tokens. Each training is conducted over 3 runs per target class.

Target Class	It.	ViT-B/16	ViT-B/32	ViT-L/16
Maltese Dog	1	32.18%	31.33%	0.13%
	2	29.86%	29.98%	0.14%
	3	33.56%	32.31%	1.23%
Bee	1	5.03%	53.39%	0.14%
	2	16.41%	43.71%	5.12%
	3	0.87%	49.77%	0.12%
Cassette	1	0.20%	22.61%	0.59%
	2	0.41%	36.95%	0.15%
	3	1.29%	14.14%	0.13%
Hockey Puck	1	10.41%	31.76%	10.64%
	2	46.42%	49.50%	5.21%
	3	25.74%	50.76%	4.24%
Pretzel	1	35.50%	51.71%	21.77%
	2	53.25%	43.11%	17.50%
	3	39.89%	51.51%	11.78%

This table reveals several interesting trends. First, generators trained on ViT-B/32 consistently outperform the ones trained on other models, across almost all target classes and iterations. ASRs are often above average, sometimes even above **50%**.

On the other hand, generators trained on ViT-L/16 show very poor performance, with most ASRs below **5%**, and often close to zero. This suggests the token-replacement approach is very ineffective on a deep model like ViT-L/16.

ViT-B/16 shows moderate performance, effective in some cases, for example for the target class *Pretzel*. However, the generators trained on target classes like *Bee* and *Cassette* have very low success rates.

We also note that the performance across iterations is more inconsistent compared to the *corner* point patch setting. To aid our comparative analysis of these two approaches, we present Figure 4.14, which illustrates how each of them performs across ViT models, averaged over all target classes.

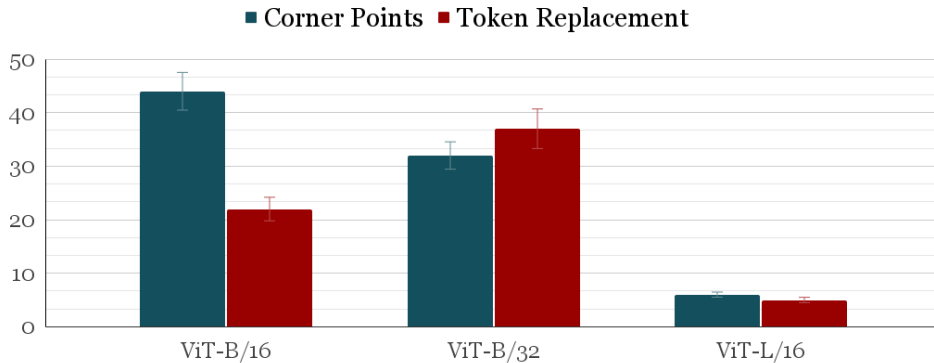


Figure 4.14 Comparison of average ASRs across victim models for corner point patches (green) and token-replacement patches (red). ASR values are averaged over five target classes.

Clearly, token-replacement strategy works the best on the ViT-B/32 model, outperforming even the results obtained from the corner-point patch method. Interestingly, even though the patches generated for ViT-B/32 in this setting looked very noisy and more pixelated (see Fig. 4.12), compared to those from previous experiments, they performed much better and achieved higher ASRs on average. This suggests that for this specific model, replacing fewer tokens entirely may be more effective than perturbing a larger number (i.e. 4 times more) of tokens partially, like in the case of the corner-point method, where four adjacent image tokens are affected (but only 25% is modified in each of them).

On the other hand, corner-point attacks yield better results for the other models. For example, generators trained on ViT-B/16 in this setting significantly outperform the ones from the token-replacement configuration. For ViT-L/16, the difference is minimal but still better for the corner-point method.

In summary, although the token-replacement approach is highly effective against ViT-B/32, it is inconsistent across other models and iterations. This highlights a lack of general reliability (just like in case of random-positioned patches). On the contrary, the corner-point strategy, while sometimes less effective on ViT-B/32, demonstrates more stability and effectiveness across all three ViT architectures. We therefore conclude that this is the most robust and dependable option overall, across all 3 mini patch configurations.

Based on these findings, we next explore the transferability of mini patch attacks.

4.3 Transferability

In addition to proposing novel attack strategies, we also investigate the transferability of mini patch attacks, that is, how well patches trained on one ViT model performs against other models.

Based on our previous experiments, the corner-point configuration consistently performed the best across all target classes and models. Additionally, it was the most reliable, with minimal variation in performance across different runs, and did not have "unlucky" generators.

Therefore, we selected patches centered at ViT corner points for our transferability analysis. We believe that their stronger and more consistent performance will provide better transferable generators and patches.

As in our earlier experiments for G-patch attacks (see section 3.4), we present transferability results for two representative target classes *Bee* and *Pretzel*. In the G-patch setting, these two target classes were chosen because the *Pretzel* class was easy to learn and generalize, whereas *Bee* was more difficult. Even though this difference is not as obvious in the mini patch experiments (since performance varies across victim models), we still keep these classes for consistency and comparability with our previous results.

Single Model Transferability

We begin with a single-model transferability configuration, where the generator is trained on a single ViT model, and then evaluated on the remaining ViT models. As in the G-patch setting, in these experiments we report:

- the average ASR across three training runs;
- the best ASR observed.

Table 4.4 presents these results on ViT models, for the two representative target classes. Diagonal entries of the table represent ASRs when the generator is evaluated on the same model it was trained on; while off-diagonal cells show how well the attack transfers to other models within the same family.

Table 4.4 Transferability results for *corner-point* patches. The table follows the format of Tab. 3.6.

Train Model	Target	ViT-B/16	ViT-B/32	ViT-L/16
ViT-B/16	Bee	<i>65</i>	0.1/ 0.2	0.4/ 0.6
	Pretzel	<i>32</i>	0.1/ 0.1	0.2/ 0.3
ViT-B/32	Bee	0.1/ 0.2	<i>38</i>	0.1/ 0.2
	Pretzel	0.1/ 0.2	<i>61</i>	0.3/ 1.5
ViT-L/16	Bee	0.1/ 0.4	0.1/ 0.2	<i>6</i>
	Pretzel	0.2/ 0.2	0.1/ 0.1	<i>10</i>

This table shows that across all training models and target classes, off-diagonal values are extremely low, often close to **0%** ASR. Interestingly, even patches trained on ViT-B/32 (32×32 in size) fail to transfer well on ViT-B/16, which showed vulnerability to much smaller, 16×16 patches.

These results indicate that mini patch attacks are very model-specific, and they are effective only on the model they are trained on. To address this, we next explore whether training on ensembles of models can produce more transferable and generalizable results.

Ensemble Transferability

In this section, we evaluate whether training on an ensemble of different models improves the transferability of mini patches. We begin by analyzing ensembles consisting of only transformer models. However, to test if architectural diversity helps, we also include Swin-B in some ensembles. We think that exposing the generator to a different transformer structure might help with generalization.

Table (4.5) summarizes ASR scores for generators trained on several combinations of ViT models. Results are, as before, reported for the same two representative classes: *Bee* and *Pretzel*.

We note here that some ensembles that contain ViT-B/32 (for example, an ensemble of ViT-B/16 and ViT-B/32) are trained on patch size 32×32 , while larger ensembles (like ViT-B/32, ViT-L/16, Swin-B and ViT-B/16, ViT-B/32, Swin-B) are trained on 16×16 patches.

This table reveals several important trends. First, training generators on ensembles generally increased ASR scores on their constituent models. For example, patches trained on the ViT-B/16, ViT-B/32 ensemble achieve success rates of **75.6%** and **89%** on ViT-B/16, as opposed to **65%** and **32%**, which were achieved when training on ViT-B/16 alone.

However, despite the improved performance on constituent models, transferability rates remain still very low. For instance, the same ViT-B/16 and ViT-B/32

Table 4.5 Transferability results of generators trained on ensembles of ViT models. Results are reported for two representative target classes Bee and Pretzel. The last column lists ASR results of the ensembles themselves. For clarity, ASR scores under models that are included in the ensemble are in italics.

Ensemble	Target	VB/16	VB/32	VL/16	Sw-B	Ens. ASR
VB/16, VB-32	Bee	<i>75.6</i>	<i>22.6</i>	0.5	0.2	55.2
	Pretzel	<i>89.0</i>	<i>45.8</i>	0.7	2.0	77.0
VB/16, VL/16	Bee	<i>39.6</i>	0.2	<i>1.9</i>	0.1	11.8
	Pretzel	<i>14.1</i>	0.1	<i>5.2</i>	0.1	7.1
VB/16, Sw-B	Bee	<i>47.5</i>	0.1	0.5	<i>80.8</i>	72.4
	Pretzel	<i>5.9</i>	0.1	0.1	<i>71.4</i>	54.0
VB/32, VL/16, Sw-B	Bee	0.3	<i>0.2</i>	<i>1.1</i>	<i>71.7</i>	18.1
	Pretzel	1.1	<i>0.1</i>	<i>10.4</i>	<i>72.7</i>	27.7
VB/16, VB/32, Sw-B	Bee	<i>34.1</i>	<i>0.2</i>	0.2	<i>75.6</i>	45.5
	Pretzel	<i>27.6</i>	<i>0.2</i>	0.3	<i>71.4</i>	42.0

ensemble achieves below **2%** ASR on ViT-L/16 and Swin-B, for both target classes. This indicates that effectiveness within the ensemble itself does not necessarily guarantee generalizability to other models.

It is interesting that some combinations of similar models, like ViT-B/16 and ViT-L/16, actually reduce effectiveness. With the overall (ensemble) ASRs **11.8%** and **7.1%**, this ensemble achieves lower ASRs on the constituent models than they achieved on their own, during training.

On the other hand, ensembles that include Swin-B usually achieve significantly higher (consistently above **70%**) success rates on Swin-B itself.

Overall, these findings suggest that while ensembling can improve performance on the models included in training, it usually does not lead to improved transferability.

Mixed-Family Ensemble Transferability

In the previous experiments, we observed very limited transferability in ViT-only ensembles, even when including structurally different transformer models like Swin-B. This drives us to next explore whether combining *fundamentally* different architectures can help create more generalizable adversarial patches. In particular, we explore mixed-family ensembles that combine Vision Transformers and CNNs.

Table 4.6 Transferability results of mixed-family ensembles on mini patches. Table follows the format of Tab. 3.14.

Ensemble	TC	VB/16	VB/32	VL/16	Sw-B	RN50	RN152	VGG
VB/16, RN50	B	<i>38.5</i>	0.2	0.6	0.2	<i>54.7</i>	1.4	0.2
	P	<i>11.4</i>	0.1	0.1	0.1	<i>58.3</i>	0.4	0.1
VB/16, VB32, RN152	B	<i>72.0</i>	<i>11.2</i>	2.6	6.7	33.3	<i>61.4</i>	0.3
	P	<i>85.2</i>	<i>49.6</i>	4.4	40.0	63.4	<i>73.6</i>	0.2
VB/32, RN101, RN152	B	0.2	<i>14.2</i>	0.1	4.0	31.1	<i>83.1</i>	0.1
	P	15.5	<i>53.0</i>	9.9	37.1	89.7	<i>94.5</i>	0.4
VB/16, RN50, RN101, RN152	B	<i>11.6</i>	0.2	0.2	0.2	<i>30.8</i>	<i>24.4</i>	0.1
	P	<i>3.8</i>	0.1	1.2	6.5	<i>59.3</i>	<i>45.5</i>	0.1

Table 4.6 presented above summarizes the results for various mixed-architecture ensembles, combining CNNs and ViTs. As we see from these results, in many cases, CNNs like ResNet50 and ResNet152 demonstrate high ASRs. For example, the ViT-B/32, ResNet101, ResNet152 ensemble achieves **83.1%** on its constituent ResNet152 model for target class *Bee*, and **94.5%** on *Pretzel*. On the other hand, we note that VGG16-BN remains consistently the most robust against every single ensemble. This was expected, as we saw similar resistance of this model in G-Patch transferability experiments.

Just like VGG16-BN, most ViTs do not benefit from mixed-architecture training. For example, ViT-B/16, ResNet50 ensemble achieves only **38.5%** ASR on ViT-B/16, compared to the **75.6%** obtained in the ViT-B/16, ViT-B/32 ensemble or even the **65%** achieved when trained on ViT-B/16 alone.

Despite these limitations, a couple of ensembles show promising transferability, especially for target class *Pretzel*. For example, the ViT-B/16, ViT-B/32, ResNet152 ensemble achieves **40%** transferability rate on Swin-B, and **63.4%** on ResNet50. Additionally, the ViT-B/32, ResNet101, ResNet152 ensemble obtains a moderate ASR of **15.5%** (although relatively high, compared to previous experiments) on ViT-B/16, **37.1%** on Swin-B and impressive **89.7%** on ResNet50.

These results suggest that both patch size and model diversity contribute to transferability. On one hand, these ensembles likely perform better because they were trained on larger, 32×32 patches. On the other hand, we can see that including CNNs in the ensembles also helped, since previous ViT-only ensembles, even when trained on larger patches, achieved much lower transferability rates.

To visualize these differences, Figure 4.15 shows examples of adversarial patches generated by the best-performing mixed ensembles.



Figure 4.15 32×32 adversarial patches generated by the best-performing mixed-architecture ensembles (ViT-B/16, ViT-B/32, ResNet152 on the left; ViT-B/32, ResNet101, ResNet152 on the right).

For comparison, we also include Figure 4.16, which presents patches from less successful ensembles (like ViT-B/16, ResNet50 and ViT-B/16, ResNet50, ResNet101, ResNet152).



Figure 4.16 16×16 adversarial patches generated by weaker mixed ensembles (ViT-B/16, ResNet50 on the left; ViT-B/16, ResNet50, ResNet101, ResNet152 on the right).

Summary

Overall, our transferability experiments reveal that corner-point patch effectiveness is highly sensitive to both model architecture and ensemble composition. Single-model training offers limited generalization, where patches often failed to transfer to different models.

While ViT-only ensembles generally improve ASR on their *constituent* models, their transferability remains still quite low. In particular, when evaluated on more robust or architecturally distinct models like ViT-L/16 or Swin-B.

On the other hand, mixed-architecture ensembles offer a modest improvement in transferability, especially when larger patches are used and CNNs like ResNet152 or ResNet101 are included. These ensembles manage to achieve higher ASRs on unseen CNNs and occasionally on ViTs.

To summarize, our findings suggest that diversity in both architecture and training data (i.e. patch size and model variety) is necessary, but not sufficient for robust transferability.

Conclusion

We conclude that the goals of this thesis were successfully achieved. We first examined the robustness of Vision Transformers against two state-of-the-art attacks: **First Gradient Sign Method** (FGSM) and **Projected Gradient Descent** (PGD). FGSM showed moderate effectiveness on ViTs ($\sim 70\%$ ASR), while CNNs appeared slightly more robust ($\sim 55\%$ ASR). PGD, on the other hand, was highly effective against both architectures, often achieving over 90% success rates. These baseline results served as a reference point for our primary investigation into patch-based adversarial attacks and their transferability across model architectures.

We then implemented the **G-patch attack** (proposed by Shao, 2024) and successfully replicated the original results. Across all experiments, our patches performed as well as (and sometimes better than) those in the paper: for ViT models we achieved $\sim 75\%$ and $\sim 90\%$ success rates (for 64×64 and 80×80 patches, respectively); and on CNNs – $\sim 80\%$ and $\sim 95\%$.

We extended this work by introducing and evaluating our own novel **Mini-Patch** attacks in three settings: *random placement*, *corner-point targeting* and *token replacement*. Based on extensive experiments, we concluded that the corner-point patch attack was the most effective and consistent, performing comparably to G-patches. While random placement may offer advantages in real-world deployment, structured *corner-point patches* provided more stable training and stronger adversarial patches. They performed especially well against some of the ViT models, achieving $\sim 60\text{--}70\%$ success rates, and $\sim 40\text{--}50\%$ on CNNs.

These findings showed a comparable level of robustness to those observed in the traditional, gradient-based attacks. While CleverHans methods like PGD remain highly effective in digital white-box settings, patch-based attacks offer a more practical and transferable alternative.

Additionally, we conducted a comprehensive **transferability** analysis for both G-patch and Mini-Patch attacks. In the G-patch setting, our findings showed that *intra-family* transferability is more effective than *inter-family* transferability. However, in inter-family transferability, mixed-ensemble training stood out as a powerful approach for generating more universal adversarial patches. From Mini-Patch attacks, we selected the *corner-point* approach, as it was the most stable and effective method across all three proposed. These experiments revealed that transferability in this setting is highly sensitive to the source model architecture and ensemble composition.

Overall, as intended, our findings contribute to a deeper understanding of both the vulnerabilities of Vision Transformer models and the transferability of adversarial attacks across architectures. We hope that this work offers important insights for future research aimed at improving model robustness against adversarial perturbations.

For future work, we point out several areas that could be expanded or improved. We recommend conducting more experiments across additional target classes and victim models, for both G-patch and Mini-patch attacks. Due to computational limits (since our experiments consumed over 500 GPU days), in some experiments we had to limit the number of iterations or victim models evaluated. Expanding

these experiments would lead to more robust and generalizable insights.

Another promising direction would be to test the patches in the physical world, for example, by deploying printed patches in real-world scenarios. This would help analyze patch effectiveness even further – beyond digital settings.

Bibliography

- ANDRIUSHCHENKO, Maksym; CROCE, Francesco; FLAMMARION, Nicolas; HEIN, Matthias, 2020. *Square Attack: a query-efficient black-box adversarial attack via random search*. Available from arXiv: 1912.00049 [cs.LG].
- BROWN, Tom B.; MANÉ, Dandelion; ROY, Aurko; ABADI, Martín; GILMER, Justin, 2018. *Adversarial Patch*. Available from arXiv: 1712.09665 [cs.CV].
- DOSOVITSKIY, Alexey; BEYER, Lucas; KOLESNIKOV, Alexander; WEISSENBORN, Dirk; ZHAI, Xiaohua; UNTERTHINER, Thomas; DEHGHANI, Mostafa; MINDERER, Matthias; HEIGOLD, Georg; GELLY, Sylvain; USZKOREIT, Jakob; HOULSBY, Neil, 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*.
- GOODFELLOW, Ian J.; SHLENS, Jonathon; SZEGEDY, Christian, 2015. *Explaining and Harnessing Adversarial Examples*. Available from arXiv: 1412.6572 [stat.ML].
- HE, Kaiming; ZHANG, Xiangyu; REN, Shaoqing; SUN, Jian, 2015. *Deep Residual Learning for Image Recognition*. Available from arXiv: 1512.03385 [cs.CV].
- KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey, 2012. ImageNet Classification with Deep Convolutional Neural Networks. *Neural Information Processing Systems*. Vol. 25. Available from DOI: 10.1145/3065386.
- MADRY, Aleksander; MAKELOV, Aleksandar; SCHMIDT, Ludwig; TSIPRAS, Dimitris; VLADU, Adrian, 2019. *Towards Deep Learning Models Resistant to Adversarial Attacks*. Available from arXiv: 1706.06083 [stat.ML].
- MORDVINTSEV, Alexander; OLAH, Christopher; TYKA, Mike, 2015. *Inceptionism: Going Deeper into Neural Networks*. Available also from: <https://research.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html>.
- NASEER, Muzammal; RANASINGHE, Kanchana; KHAN, Salman; KHAN, Fahad Shahbaz; PORIKLI, Fatih, 2022. *On Improving Adversarial Transferability of Vision Transformers*. Available from arXiv: 2106.04169 [cs.CV].
- PAPERNOT, Nicolas; FAGHRI, Fartash; CARLINI, Nicholas; GOODFELLOW, Ian; FEINMAN, Reuben; KURAKIN, Alexey; XIE, Cihang; SHARMA, Yash; BROWN, Tom; ROY, Aurko; MATYASKO, Alexander; BEHZADAN, Vahid; HAMBARDZUMYAN, Karen; ZHANG, Zhishuai; JUANG, Yi-Lin; LI, Zhi; SHEATSLEY, Ryan; GARG, Abhibhav; UESATO, Jonathan; GIERKE, Willi; DONG, Yinpeng; BERTHELOT, David; HENDRICKS, Paul; RAUBER, Jonas; LONG, Rujun, 2018. Technical Report on the CleverHans v2.1.0 Adversarial Examples Library. *arXiv preprint arXiv:1610.00768*.
- RECHT, Benjamin; ROELOFS, Rebecca; SCHMIDT, Ludwig; SHANKAR, Vaishaal, 2019. *Do ImageNet Classifiers Generalize to ImageNet?* Available from arXiv: 1902.10811 [cs.CV].
- SHAO, Mingzhen, 2024. Designing Physical-World Universal Attacks on Vision Transformers. In: *Neurips Safe Generative AI Workshop 2024*. Available also from: <https://openreview.net/forum?id=DqBPk7887N>.

SIMONYAN, Karen; ZISSERMAN, Andrew, 2015. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. Available from arXiv: 1409.1556 [cs.CV].

List of Figures

1.1	Representation of the architecture of a Convolutional Neural Network. Source: Wikimedia.	10
1.2	Representation of the architecture of a Vision Transformer. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence (from Dosovitskiy et al., 2021).	12
1.3	Visualization of the adversarial perturbations and examples found by the l_{inf} and l_2 versions of the Square Attack on ResNet-50 (Andriushchenko et al., 2020).	13
1.4	A real-world attack on VGG16, using a physical patch generated by the white-box ensemble method. When a photo of a tabletop with a banana and a notebook (top photograph) is passed through VGG16, the network reports class ‘banana’ with 97% confidence (top plot). If we physically place a sticker targeted to the class “toaster” on the table (bottom photograph), the photograph is classified as a toaster with 99% confidence (bottom plot) (Brown et al., 2018).	14
2.1	Original image with the true label “Barometer” (left), and the corresponding adversarial image that fooled model ViT-B/16 into classifying it as “Magnetic Compass” (right). The attack was conducted with the epsilon value $\epsilon = 0.2$	19
2.2	Original images (left), and their corresponding adversarial images that failed to mislead victim model ResNet50 (right). The attack was conducted with the epsilon value $\epsilon = 0.2$, for a target class “Cassette”.	21
2.3	Original images (left), and their corresponding adversarial images that fooled the victim model ViT-B/32 into misclassifying the images as “Cassette” (left) and “Bee”(right). The attack was conducted with both epsilon values, $\epsilon = 0.05$ (image “Church”), and $\epsilon = 0.2$ (image “Dog”).	24
2.4	Attack success rates in targeted and untargeted settings, for both epsilon values. The ASR values for targeted attacks are averaged over all five target classes.	24
3.1	Overview of the G-Patch generating model. Source: Shao, 2024. .	25
3.2	Training metrics of a generator creating 64×64 adversarial patches. The discriminator model is ViT/B-16; target class is “Maltese Dog”. .	30
3.3	Target class “Bee”; Victim Model ViT/B-16. Initial random noise vector (left); adversarial patch after the first epoch (middle) and the final, best-performing epoch (right).	30

3.4	Best-performing adversarial patches for the target class "Bee" across six different victim models: ViT-B/32, ViT-L/16, Swin-B, ResNet50, ResNet152, and VGG16-BN (from left to right, and top to bottom). Each patch is the result of a separate generator trained to attack the respective model.	31
3.5	Perturbed images that were misclassified by the victim model ResNet152 as the target class <i>Bee</i> . Patch generator was trained on the same discriminator model. Original image labels were: "Elephant", "Tree Frog" from left to right, respectively. This figure also demonstrates the mild difference between the patches from different runs of the same experiment.	31
3.6	Average ASR per target class and model. Shows model vulnerability to G-patch attacks across all classes.	32
3.7	Average ASR per target class (robust models only). Highlights class-wise patch generalizability, excluding easiest-to-fool models.	33
3.8	Best-performing adversarial patches for the target class "Maltese Dog" across six different victim models: ViT-B/16, ViT-B/32 and ViT-L/16 (top row from left to right); and ResNet50, ResNet152, VGG16-BN (bottom row from left to right).	34
3.9	Best-performing adversarial patches for the target class "Maltese Dog" across 4 iterations for the victim model ViT-B/32.	34
3.10	Example outputs from the "Inceptionism" study (Mordvintsev et al., 2015), including visualizations for images <i>Starfish</i> , <i>Banana</i> , <i>Parachute</i> , and <i>Anemone Fish</i> (from left to right).	35
3.11	Example adversarial images that were misclassified by a victim model Swin-B. Actual labels of the images were "ATM", "Bluetick Coonhound" and "Stopwatch" from left to right, respectively. The patch generator was trained on ViT-B/16.	35
3.12	Average ASR per target class and model. Shows model vulnerability to 80×80 sized G-patch attacks across all classes.	36
3.13	Average ASR values across all target classes for the victim model ViT-L/16. Brown bars represent ASR values of 64×64 patches, pink ones – of 80×80	36
3.14	Average ASR values across all target classes for the victim model ResNet152. Green bars represent the ASR values achieved by 64×64 patches, blue ones – by 80×80	37
3.15	Attack success rates of generators trained on different ViT ensembles and evaluated on the held-out model. The plot shows results for both target classes (<i>Bee</i> and <i>Pretzel</i>) and patch sizes (64×64 and 80×80).	42
3.16	Comparison of 80×80 adversarial patches generated for target class <i>Bee</i> . The top row shows patches trained on individual models (ViT-B/16, ViT-B/32, ViT-L/16, Swin-B). The bottom row shows patches trained on ensembles that exclude the corresponding model from the top row (for example, the first patch in the second row was trained on an ensemble of ViT-B/32, ViT-L/16, Swin-B models).	42

3.17	Adversarial patches of size 80×80 and target class <i>Bee</i> , trained on mixed-ensembles. The order of patches follows the list of ensembles above, from left to right, and top to bottom.	46
3.18	Adversarial patches of size 80×80 and target class <i>Pretzel</i> , trained on mixed-ensembles. The order of patches follows the format of Fig. 3.17 above.	47
3.19	ASR values against ViT target models for adversarial patches generated from mixed-family ensembles using 80×80 patches and target class <i>Pretzel</i>	48
3.20	ASR values against CNN target models for adversarial patches generated from mixed-family ensembles using 80×80 patches and target class <i>Pretzel</i> . Note, that ASR values for ensembles that included the target model are not present in the chart.	49
4.1	Average ASR during training of a generator against ViT-B/32 (target class: Maltese Dog).	52
4.2	Comparison between unsuccessful (left: epoch 1 and 16) and successful (right: epoch 1 and 38) patches for ViT-B/32 targeting "Maltese Dog". Both were trained under the same settings.	53
4.3	Example adversarial images that were misclassified as the target class "Bee". Original labels were: <i>Gazelle</i> , <i>Green Mamba</i> , <i>Welsh Corgi</i> , <i>Bear</i> (left to right). The first two images (Gazelle and Green Mamba) display 16×16 adversarial patches trained on Swin-B; whereas the last two (Welsh Corgi and Bear) demonstrate 32×32 patches trained on ViT-B/32.	53
4.4	Comparison of average ASRs across victim models for Mini Patch (blue), G-Patch 64×64 (purple), and G-Patch 80×80 (gold) attacks. ASR values are averaged over five target classes. For Mini Patch, error bars indicate the standard deviation of the average ASR across those five target classes, highlighting variability in per-class performance.	54
4.5	Adversarial patch of size 32×32 px. centered at a <i>corner</i> point of the image (red lines do not represent the exact tokenization of ViT models, for the sake of better visualization, tokens here are of size 56×56). This results in attacking 4 tokens with 8×8 size adversarial patches.	55
4.6	Average ASR of randomly placed mini patches; per target class and model.	56
4.7	On the left: patches from two different runs in the random placement setting; on the right: patches from two different run in the <i>corner points</i> scenario. All generators were trained on the victim model ViT-B/32, for target class "Bee". All patches are of size 32×32	57
4.8	Example adversarial images that were misclassified as the target class "Bee". Original labels were: <i>Monkey</i> , <i>Spider</i> (left to right). Patches shown on these images were created by a generator trained on ViT-B/32.	57

4.9	Comparison of average ASRs across victim models for <i>random placement</i> patches (blue) and <i>corner point</i> patches (purple). ASR values are averaged over five target classes. Error bars indicate the standard deviation of the average ASR across those five target classes, highlighting variability in per-class performance.	58
4.10	Comparison of average ASRs across victim models for <i>random placement</i> (blue) and <i>corner point</i> patches (green), to G-Patch 64×64 (purple), and 80×80 (gold) attacks.	58
4.11	Adversarial patch placed entirely inside the token. Red lines do not represent the exact tokenization of ViT models, for the sake of better visualization, tokens here are of size 56×56 (and so is the example adversarial patch). This results in replacing 1 token entirely.	59
4.12	Adversarial patches trained on ViT-B/16, ViT-L/16 and ViT-B/32 (from left to right; the last two are patches from different runs on ViT-B/32 model).	60
4.13	Example adversarial images that were misclassified as the target class "Bee". The first two images show patches that were created by a generator trained on ViT-B/16 model, while the last two – on ViT-B/32.	60
4.14	Comparison of average ASRs across victim models for corner point patches (green) and token-replacement patches (red). ASR values are averaged over five target classes.	61
4.15	32×32 adversarial patches generated by the best-performing mixed-architecture ensembles (ViT-B/16, ViT-B/32, ResNet152 on the left; ViT-B/32, ResNet101, ResNet152 on the right).	65
4.16	16×16 adversarial patches generated by weaker mixed ensembles (ViT-B/16, ResNet50 on the left; ViT-B/16, ResNet50, ResNet101, ResNet152 on the right).	65

List of Tables

2.1	Attack success rates of untargeted FGSM attack on ViT and CNN models, using two different values of epsilon.	20
2.2	Attack success rates of the FGSM attack on ViT and CNN models in the targeted setting, evaluated across five target classes and for the epsilon value $\epsilon = 0.05$	21
2.3	Attack success rates of the FGSM attack on ViT and CNN models in the targeted setting, evaluated across five target classes and for the epsilon value $\epsilon = 0.2$	21
2.4	Attack success rates of untargeted PGD attack on ViT and CNN models, using two different values of epsilon.	22
2.5	Attack success rates of the PGD attack on ViT and CNN models in the targeted setting, evaluated across five target classes and for the epsilon value $\epsilon = 0.05$	23
2.6	Attack success rates of the PGD attack on ViT and CNN models in the targeted setting, evaluated across five target classes and for the epsilon value $\epsilon = 0.2$	23
3.1	ASR scores, evaluated over 5 runs per target class. Target models are abbreviated for compactness (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50).	32
3.2	Average ASR for 80×80 patches across five runs per target class. Target models are abbreviated for compactness (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50, etc.).	35
3.3	Average ASR across all five target classes, compared with the ASR values reported in the original paper.	38
3.4	ASR results for each target class using 64×64 adversarial patches on ViT models, compared with the ASR values reported in the original paper.	38
3.5	ASR results for each target class using 80×80 adversarial patches on ViT models, compared with the ASR values reported in the original paper.	38
3.6	Transferability results for 64×64 size patches. Each cell contains two values separated by a slash (avg/best): the average ASR across five runs (left) and the best ASR observed (right). Diagonal entries represent same-model evaluation; off-diagonal entries show transferability.	40
3.7	Transferability results for 80×80 adversarial patches across ViT models. Format follows Table 3.6 above.	40
3.8	Transferability results of a generator trained on ensembles of three ViT models and tested on the remaining one, using 64×64 adversarial patches. Model names are shortened for compactness; and the last column, under "Ens. ASR", lists ASR results of the ensembles themselves.	41
3.9	Transferability results of a generator trained on ViT ensembles using 80×80 patches. Format follows Table 3.8.	41

3.10	ASR results for 64×64 adversarial patches transferred from ViT models to CNNs. Each cell shows average/best ASR across five runs. The ASRs of best-performing generators are in bold for more clarity.	44
3.11	ASR results for 80×80 adversarial patches transferred from ViT models to CNNs. Format follows Table 3.10.	44
3.12	Transferability results of generators trained on ensembles of ViT models and tested on CNNs, using 64×64 adversarial patches. Results are reported for two representative target classes <i>Bee</i> and <i>Pretzel</i> . The last column lists ASR results of the ensembles themselves.	45
3.13	Transferability results of generators trained on ensembles of ViT models and tested on CNNs, using 80×80 adversarial patches. Table follows the format of Tab. 3.12 above.	45
3.14	Transferability results of 64×64 patches trained on a joint ViT+CNN ensemble. Target class names are shortened for compactness: "B" stands for "Bee" and "P" for "Pretzel". Additionally, for clarity, ASR scores under models that are included in the ensemble are in <i>italics</i>	47
3.15	Transferability results of 80×80 patches trained on a joint ViT+CNN ensembles. Table follows the format of Tab. 3.14 above.	48
4.1	ASR scores, evaluated over 3 runs per target class. Target models are abbreviated for compactness (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50).	52
4.2	ASR scores, evaluated over 3 runs per target class. Target models are abbreviated for compactness (e.g., VB/16 for ViT-B/16, RN50 for ResNet-50).	56
4.3	ASR scores for adversarial patches replacing ViT tokens. Each training is conducted over 3 runs per target class.	61
4.4	Transferability results for <i>corner-point</i> patches. The table follows the format of Tab. 3.6.	63
4.5	Transferability results of generators trained on ensembles of ViT models. Results are reported for two representative target classes Bee and Pretzel. The last column lists ASR results of the ensembles themselves. For clarity, ASR scores under models that are included in the ensemble are in <i>italics</i>	64
4.6	Transferability results of mixed-family ensembles on mini patches. Table follows the format of Tab. 3.14.	64

A Attachments

The electronic attachment to this thesis contains the full implementation of the adversarial attack experiments described throughout the thesis. The project directory consists of the following:

A.1 CleverHans

This directory contains scripts for evaluating classical adversarial attacks using the **CleverHans** library. It includes:

- Scripts for running attacks such as FGSM, PGD.
- Support for multiple models including ResNet50, ViT-B/16, and ViT-L/32.
- Logging of results to Weights & Biases.

Additionally, this directory contains a detailed developer documentation at `docs/build/html`.

A.2 Random_Position_Patch

This directory contains the implementation of **G-Patch** and **Mini-Patch** attacks. It includes:

- `main.py` – main script for training and evaluating adversarial patches.
- `generator.py` – patch generator architectures.
- `deployer.py` – handles patch deployment and image manipulation.
- `loss.py` – supporting code for training and evaluation.
- `Mini_Deployer` – directory that provides a modified deployer mechanism for Mini-Patch attacks.
- `Generator` – directory containing 4 pre-trained generators that can be loaded and used for test experiments.

Additionally, this directory contains a detailed developer documentation at `docs/build/html`.

A.3 User Documentation

A `README.md` file with detailed user documentation that explains how to run experiments in CleverHans and Random Position Patch frameworks.

A.4 Link to the GitHub Repository

The full source code and documentation are also available on GitHub at: <https://github.com/TamarNoselidze/Thesis>.